Computer Networks Research Project

Phase 2: Extended Literature Review and Research Methodology

Decision Horizons in Graph-Based Anti-Money Laundering: What Detectors Know, When They Decide, and Whether It Is Early Enough

Developed from the Phase 1 topic: Directed Multigraph GNNs for Real-Time AML on Transaction Networks

---

Submitted by:

Saif Shahzad (23I-2634)
Shahoud Shahid (23I-2515)

Program: BS Data Science
Department: Artificial Intelligence and Data Science

Course: Computer Networks
Semester: 7
Academic Year: 2026
Submission Date: October 20, 2026
Revision 2: October 25, 2026
Revision 3: November 1, 2026

National University of Computer and Emerging Sciences (FAST-NUCES)

Contents

1 Introduction and Project Context **6**

2 Connection to the Phase 1 Literature Review **7**
2.1 Phase 1 Core Research Gaps . . . 8
2.2 How Phase 2 Refines the Phase 1 Gaps . . . 8
2.3 Phase 2 Expansion Scope . . . 8

3 Literature Search, Screening and Verification Protocol **8**
3.1 Search Strategy . . . 8
3.2 Inclusion Criteria . . . 9
3.3 Verification Procedure . . . 9
3.4 Included Papers . . . 10
3.5 Screened but Excluded Candidates . . . 10

4 Detailed Individual Paper Analyses **10**
4.1 Paper 1: GeneralDyG — A Generalizable Anomaly Detection Method in Dynamic Graphs . . . 10
4.2 Paper 2: Tian & Zhang — Continual Graph Learning with Knowledge-Augmented Replay . . . 13
4.3 Paper 3: Ferretti et al. — Enhancing AML Frameworks in Cryptocurrency Transactions . . . 16
4.4 Paper 4: TFGAT — Global-Local Graph Attention with Cyclic Pseudo-Labels . 18
4.5 Paper 5: Kasula et al. — Blockchain Address Classification Using AJK-GraphSAGE 21
4.6 Paper 6: LMAE4Eth — Ethereum Fraud Detection with Transaction Semantics and Masked Graph Embedding . . . 23
4.7 Paper 7: ChronoWave-GNN — A Wavelet-Temporal Graph Transformer for Bitcoin AML . . . 25
4.8 Paper 8: Al-E'mari et al. — Explainable GNNs for Multi-Class Fraudulent Address Classification . . . 28
4.9 Paper 9: TAML — Translating Latent Representations for Money Laundering Detection . . . 30

5 Closest Prior Work and Methodological Positioning **32**
5.1 Detailed Review of Closest Prior Work . . . 32
5.1.1 Multi-GNN: Provably Powerful GNNs for Directed Multigraphs (Egressy et al., AAAI 2024) . . . 32
5.1.2 DIAM: Illicit Account Detection on Cryptocurrency MultiGraphs (Ding et al., CIKM 2024) . . . 33
5.1.3 MEGA-GNN: Message Passing for Multigraphs with Edge Attributes (Bilgi & Atasu) . . . 33
5.1.4 Graph Feature Preprocessor (Blanuša et al., ICAIF 2024) . . . 33
5.1.5 FraudGT: the Current Accuracy Reference on IBM AML (Lin et al., ICAIF 2024) . . . 34
5.1.6 SALT-GNN: Dense Neighbourhoods in AML Graphs (Losavio et al., arXiv 2026) . . . 34
5.1.7 Batch Staleness in Memory-Based Temporal GNNs . . . 34
5.1.8 What Published AML Numbers Measure: Evaluation Views . . . 35
5.1.9 Adversarial Robustness of Transaction and Temporal Graph Models . . . 36
5.1.10 Delay, Money and Alert Capacity . . . 37

5.1.11 TeMP-TraG and Amatriciana . . . 37
5.2 Comparative Methodological Positioning Matrix . . . 38
5.3 Synthesis: The Defensible Scientific Niche . . . 38

6 Thematic Extended Literature Review and Synthesis 39
6.1 Theme 1: Evolution of Temporal Graph Modeling Paradigms . . . 39
6.2 Theme 2: Graph Structure vs. Tabular Domain Features . . . 39
6.3 Theme 3: Multigraph Preservation vs. Topological Simplification . . . 40
6.4 Theme 4: Extreme Class Imbalance and Optimization Pitfalls . . . 40
6.5 Theme 5: Methodological Rigor, Temporal Leakage, and Systems Latency . . . 41

7 Comprehensive Cross-Paper Comparison Table 41
7.1 Cross-Study Capability Matrix . . . 45

8 Research Gap Identification and Synthesis 45
8.1 Detailed Justification of the Four Research Gaps . . . 45

9 Refined Research Gap and Problem Formalization 48
9.1 Systematic Six-Question Justification . . . 48
9.2 Formal Project Statements . . . 49

10 Proposed Research Methodology: The 12-Step Framework 51
10.1 Step 1: Define the Research Input . . . 51
10.2 Step 2: Data Collection . . . 52
10.3 Step 3: Data Preprocessing . . . 52
10.4 Step 4: Feature Engineering and Representation . . . 53
10.5 Step 5: Proposed Methods . . . 54
10.6 Step 6: Baseline Methods and Controls . . . 55
10.7 Step 7: Experimental Design . . . 56
10.8 Step 8: Evaluation Metrics . . . 56
10.9 Step 9: Statistical and Comparative Analysis . . . 57
10.10 Step 10: Component and Ablation Analysis . . . 58
10.11 Step 11: Error and Failure Analysis . . . 58
10.12 Step 12: Final Research Outputs . . . 59

11 Consolidated Methodology Summary Table 59

12 Research Methodology Architecture Diagram 62
12.1 Architectural Narrative and Operational Band Dynamics . . . 62

13 Project Execution Plan, Feasibility, and Risk Management 62
13.1 Milestones . . . 62
13.2 Compute Strategy . . . 62
13.3 Risk Assessment and Mitigation Strategies . . . 65

14 Preliminary Evidence (1 November 2026) 65
14.1 What Has and Has Not Been Run . . . 65
14.2 The Benchmark Tail . . . 65
14.3 Quality Against Horizon . . . 68
14.4 F1 and Stoppable Value Disagree . . . 68
14.5 Adaptive Alerting: Incomplete . . . 70
14.6 Hypothesis Status . . . 71

15 Publication Potential **71**
15.1 Novelty and Scientific Contribution Assessment 71
15.2 Target Publication Venues 72

16 Conclusion **73**

A Requirement Traceability Matrix **73**

B Literature Verification Log **74**

List of Figures

1 Research methodology architecture of the decision-horizon study. The twelve steps are organised in six bands: (A) inputs and task definition (Steps 1–2), (B) preprocessing and representation (Steps 3–4), (C) proposed methods and baselines (Steps 5–6), (D) experiments (Step 7), (E) evaluation and analysis (Steps 8–11), and (F) research outputs (Step 12). Dashed arrows denote the view definition and the frozen hypotheses, which constrain later stages without passing data. . . 63
2 HI-Small by day. After the documented 10-day span, daily volume falls by about three orders of magnitude, while the illicit share rises to about 60%. Most of the test split's days lie in this region. 65
3 Quality and stoppable value against decision horizon, HI-Small, Released protocol. The two panels use separate axes because the measures differ in scale. Error bars show one s.d. over seeds. Table 14 gives the values. 69

List of Tables

1 Evolution of the Phase 1 Research Gaps into the Refined Phase 2 Gaps 9
2 Eligibility Audit of the Nine Phase 2 Papers (all metadata verified against Crossref) 10
3 Screened Candidates Excluded from the Phase 2 Quota 11
4 Positioning of This Project Against the Closest Prior Work 38
5 Cross-Paper Comparison of the Nine Phase 2 Papers (values as reported in the full texts examined) 42
6 Capability Matrix of the Reviewed Studies, the Closest Prior Work, and This Project 45
7 Research-Gap Synthesis Table (evidence from three primary 2025–2026 papers) . 46
8 Consolidated 12-Step Research Methodology Pipeline 60
9 Milestones and Decision Gates (2026–2027). Dates after 15 November are planning targets; venue dates are in Section 15. 64
10 Compute Plan by Experiment. GPU hours are estimates except where marked; the only published anchor is about 3.6 h for one Multi-GNN run on an RTX 3090 [1]. 64
11 Risk Assessment and Mitigation Strategy Matrix 66
12 Status of Experiments 67
13 The post-simulation tail on AMLworld. The date rule flags every transfer in the tail. *It is a diagnostic, not a detector*: it has no learning, and it uses knowledge of the tail. Published values are from Egressy et al. [1], not re-run. 67
14 Horizon sweep, HI-Small, Released protocol (LightGBM, mean over 3 seeds, ± s.d.). SV_B is mean stoppable value at B alerts/day with thresholds set on validation. Detection is at $B = 500$. "Core" removes the tail. Attempts: 86. . . . 68
15 Stoppable value at 500 alerts/day under two threshold rules, HI-Small, Released protocol. "Val." sets thresholds on validation, as a deployment would. "Equal alerts" gives every horizon the same alert count on the test window, as a sensitivity analysis. "Transfers" is the transfer-count analogue of SV. 69
16 Stoppable value at 500 alerts/day, HI-Small, Documented protocol. Only horizons with all three seeds complete are shown; the run was stopped before 6 h. 70
17 Detection and stoppable value by typology at 500 alerts/day, HI-Small. Each typology has few attempts, so differences between typologies are indicative only. 70
18 Adaptive alert policy against the best fixed horizon selected on validation, HI-Small, Released protocol. *One seed only; the run was stopped.* 70
19 Frozen Hypotheses and Current Evidence 71
20 Phase 2 Requirement Traceability Matrix 73

21 Literature Verification Log 75

1 Introduction and Project Context

Public blockchains such as Bitcoin and Ethereum settle value transfers between pseudonymous accounts without intermediaries. The same properties that make them attractive (global reach, irreversibility and permissionless participation) also make them attractive for laundering criminal proceeds. Chainalysis estimates that illicit cryptocurrency addresses received *at least* US$154 billion in 2025, which the company explicitly describes as a lower-bound estimate [2].

Rule-based anti-money laundering (AML) monitoring relies largely on per-account thresholds and static heuristics (aggregate inflow, velocity, watch-lists). Laundering, however, is a *relational and temporal* activity that is deliberately spread across many accounts and many transfers. Typical patterns include:

1. **Fan-out / fan-in and scatter–gather:** funds are split across many intermediary accounts and later re-aggregated, often through many small parallel transfers. These typologies (together with cycles, bipartite, stack and random patterns) are exactly those simulated in the IBM synthetic AML benchmark [3].

2. **Peeling chains (UTXO chains):** a large unspent output is repeatedly split, sending a small amount onward and returning change, producing long, directed chains of transactions.

3. **Bursty layering:** many transfers between the same pair of accounts within a short interval, which a static, per-pair summary ("total amount sent from u to v") hides.

Graph Neural Networks (GNNs) [4–6] learn from such relational structure. When applied to blockchains and bank transfer networks, nodes are accounts (or transactions) and edges are timestamped, valued transfers. Our extended review of nine 2025–2026 studies (Sections 4–7) and of the closest earlier work (Section 5) shows three recurring, evidence-backed limitations:

1. **Lossy multigraph collapse.** Several recent pipelines merge all transfers between an account pair into a single edge before learning: Tian and Zhang sum the amounts of parallel edges [7], Al-E'mari et al. remove duplicate transaction edges [8], and the IJCAI-26 TAML system defines one edge per account pair whose features aggregate all transactions within a time span [9].

2. **Evaluation views that do not match deployment.** Strong detectors are evaluated with a view of the transaction stream that a real-time system would not have. On the most widely used AML benchmark, the standard protocol gives each test transfer access to transactions that happen after it (Section 5.1.8). A temporal split separates training from testing; it does not make test-time inference causal.

3. **Evaluation protocols that hinder comparison.** Random or stratified node splits [8,10,11], undocumented splits and baselines copied from other papers [12], and balanced re-sampling [7] make reported numbers difficult to compare and can overstate deployable performance.

These limitations must be read against important earlier work. Some of it already handles directed multigraphs offline (Multi-GNN [1], DIAM [13], MEGA-GNN [14]) or streams hand-engineered subgraph features in real time (Graph Feature Preprocessor [15]). Other work studies, in generic temporal-graph settings, the staleness that batching causes in memory-based models (PRES [16], BADGNN [17], AnchorRM [18], StreamTGN [19]).

Our contribution changes the question from *how accurate is a detector* to *what does a detector know when it decides, and is that decision early enough to matter*. It has four parts. The first is the **decision horizon** H: how long after a transfer a detector may look before deciding. The horizon places real-time scoring ($H = 0$), batch monitoring (finite H) and the standard benchmark protocol ($H = \infty$) on one axis. The second is an **audit of the AMLworld benchmark**: test metrics on all four Small and Medium datasets include a period

after background traffic has ended, which materially raises headline F1. The third is an **outcome metric**, the share of laundered value not yet moved when the first alert fires, together with an adaptive multi-horizon alert policy that spends a daily alert budget across decision points. The fourth is a **timing-only threat model** (an adversary who changes nothing except when real transfers are submitted) and a design-level defence, **timing-invariant detection**, which builds on evidence such an adversary cannot alter: logical-clock windows, event order and conservation of funds.

Status of claims. This document separates four kinds of statement: *verified* facts about published work or code, read directly from the source; *measured* results, generated from our logs and labelled with their dataset; *hypotheses*, stated before the data that tests them; and *proposals*. Results from HI-Small are discovery results. They become confirmed claims only on datasets not yet modelled when the hypotheses were frozen (Section 14).

Revision note (Revision 3, 1 November 2026). This revision keeps what the nine paper analyses (Section 4) report about each paper unchanged, and updates only the sentences that link them to our design. A direct audit of the Multi-GNN and FraudGT code and of the raw data found problems with Revision 2's research design, so this revision (i) replaces the primary research question: Revision 2's staleness penalty assumed a unit-based detector may not use its own unit, a rule inherited from TGN's semantics rather than from causality; (ii) retires the intra-block context as a claimed contribution, because AnchorRM covers the learned version and a detector alerting at unit close may legitimately use the whole unit; (iii) makes DMT-GNN's memory and two-stage aggregation a gated secondary question; (iv) replaces the 21-day plan, because the instructor has confirmed that 15 November 2026 is a progress milestone rather than the final deadline; and (v) reports preliminary measured results. Revision 2 is preserved in the project archive.

Why this is a Computer Networks project. A blockchain or instant-payment rail delivers transactions to observers as a stream, and an AML detector is a stream-processing service attached to that network. The central trade-off of this work is the classic networking trade-off between *delay and information*: waiting longer delivers more evidence but a later decision. It is closely related to the freshness questions studied through Age of Information [20], with one difference: here the value of an update depends on an adversary's funds-flow schedule. The proposed defence draws directly on logical clocks from distributed systems [21]. We therefore measure end-to-end decision delay as the sum of the chosen horizon and of queueing and service latency under offered load, using measured service times and a queue simulator validated against the M/M/1 closed form. The regulatory motivation is concrete: EU Regulation 2024/886 requires euro credit transfers to settle within ten seconds at all hours [22, 23], which leaves no overnight batch window.

2 Connection to the Phase 1 Literature Review

In Phase 1 we established the topic *"Directed Multigraph GNNs for Real-Time AML on Cryptocurrency Transaction Networks"* and critically analysed three 2026 works. These three papers are *not* counted towards the Phase 2 quota.

1. **TcfDyG [24]** (*Blockchain: Research and Applications*, 2026, Art. no. 100553): a continuous-time dynamic GNN with time-aware neighbour sampling, multi-channel feature encoding and MLP-Mixer channel fusion, evaluated mainly on temporal *link prediction* (including Bitcoin-Alpha, Bitcoin-OTC and Ethereum phishing interaction graphs). Limitations noted in Phase 1: link-prediction framing rather than transaction-level AML decisions, local sampling, and no streaming latency analysis.

2. **ETX2Vec [25]** (*Scientific Reports*, 16, Art. no. 16743, 2026): temporal, amount-biased random walks with Skip-Gram embeddings on an Ethereum transaction multigraph that keeps timestamps and amounts on edges. Limitations: no GNN backbone, transductive and offline, one-hop subgraphs.

3. **FG-EGCN [26]** (*Scientific Reports*, 16, Art. no. 22352, 2026): an EvolveGCN-H temporal encoder with a residual feature branch, node-wise feature gating and focal loss on Elliptic. Limitations: discrete snapshots, no edge-level multiplicity, online updating left to future work.

2.1 Phase 1 Core Research Gaps

The Phase 1 synthesis identified four gaps:

- **Gap 1 (Structural):** no integrated directed-multigraph temporal GNN that preserves parallel transfers, fund-flow direction and continuous time.
- **Gap 2 (Operational):** no end-to-end streaming evaluation (latency, throughput, online memory updates).
- **Gap 3 (Generalization):** reliance on single or synthetic benchmarks without cross-domain validation.
- **Gap 4 (Forensic):** insufficient explanations for investigators.

2.2 How Phase 2 Refines the Phase 1 Gaps

Phase 2 preserves the research topic, motivation and direction of Phase 1. The extended literature (Sections 4–8) changes *how* each gap is framed, as summarized in Table 1. The main change is that "a GNN that handles directed multigraphs" is no longer an open problem in the *offline* setting, and that the evaluations establishing this assume a view of the future that real-time deployments do not have. The defensible gap is therefore what a detector knows when it decides, and whether that decision is early enough to matter, with multigraph-aware aggregation inside continuous-time memory as a gated extension.

2.3 Phase 2 Expansion Scope

Phase 2 analyses **nine additional papers** published in 2025–2026 in Q1/Q2 journals or CORE A* conferences (Section 3). They are complemented by a positioning study of the closest earlier work, which is outside the 2025–2026 window and therefore not counted (Section 5).

3 Literature Search, Screening and Verification Protocol

3.1 Search Strategy

Revision 3 repeated the searches on 1 November 2026, adding terms for look-ahead and temporal leakage in AML evaluation, early detection and time to detection, timing or timestamp attacks on temporal graphs, and the AMLworld date range; the additions are logged in Appendix B. The original searches were run in October 2026 across Crossref, IEEE Xplore, SpringerLink/Nature, MDPI, ScienceDirect, the AAAI and IJCAI proceedings, and arXiv (arXiv was used only to locate authors' versions of peer-reviewed papers or closest-prior-work preprints). Query terms combined {*anti-money laundering, illicit transaction, phishing, fraud*} with {*graph neural network, temporal graph, dynamic graph, multigraph*} and {*Bitcoin, Ethereum, cryptocurrency, blockchain, financial transaction network*}.

Table 1: Evolution of the Phase 1 Research Gaps into the Refined Phase 2 Gaps

Phase 1 gap	**What the Phase 2 evidence shows**	**Phase 2 status**
Gap 1 (Structural)	Offline multigraph GNNs exist [1,13,14], but recent 2025–2026 AML pipelines still collapse parallel transfers [7–9], and no reviewed work combines multi-edge aggregation with continuous-time memory.	Refined into **G1** (Section 8); now a gated extension.
Gap 2 (Operational)	Per-batch latency is sometimes reported [11,27], and high offline throughput exists [1]; batch staleness is studied for generic temporal graphs [16,18,19]. What is missing for AML is detection quality and stoppable value as functions of an explicit decision horizon; published evaluations use an unstated look-ahead (Section 5.1.8).	Refined into **G2**; the primary research question and the main networking contribution.
Gap 3 (Generalization)	Random splits, balanced re-sampling and single synthetic benchmarks dominate [7,9–11].	Refined into **G3**: one chronological, explicit-view protocol across several datasets. The AMLworld audit adds **G5**, and the threat model adds **G6**.
Gap 4 (Forensic)	Post-hoc explanation on static graphs is now evaluated rigorously [8], but explaining memory-based temporal models is a separate research problem.	Retained as **G4**, *explicitly deferred*: qualitative case studies only.

3.2 Inclusion Criteria

A paper was counted towards the Phase 2 quota only if *all* of the following held:

1. published (final version of record) in 2025 or 2026;
2. an original research article (not an editorial, survey, news item or web page);
3. published in a venue ranked Q1/Q2 (SJR or JCR) or a CORE A*/A conference;
4. directly relevant to learning on cryptocurrency or financial transaction graphs for illicit-activity detection;
5. with an identifiable method, dataset and evaluation protocol;
6. with a full text that we could read in order to verify the details reported in Section 4.

3.3 Verification Procedure

For each paper we (i) verified title, authors, venue, volume/pages or article number, year and DOI against Crossref metadata and the publisher's landing page; (ii) read the full text (publisher version, or the authors' version where the publisher page was not machine-readable) and checked every number reported in this document against it; and (iii) checked the venue ranking against the CORE portal (conferences) or SJR/JCR-derived listings (journals). The SCImago portal itself blocked automated access during this review. Journal quartiles were therefore taken from publisher bibliometric pages and secondary SJR/JCR mirrors, and are labelled as such in Appendix B. Where a detail could not be verified, it is either omitted or explicitly marked as "not reported in the version examined".

3.4 Included Papers

Table 2 lists the nine included papers.

Table 2: Eligibility Audit of the Nine Phase 2 Papers (all metadata verified against Crossref)

#	First author / short title	Year	Venue (publisher)	Ranking	Relevance to this project
P1	X. Yang — GeneralDyG [28]	2025	Proc. AAAI-25 (AAAI)	CORE2023 A*	Event-level temporal ego-graph anomaly detection; evaluated on Bitcoin-Alpha/OTC trust networks.
P2	Z. Tian — KAR [7]	2025	Electronics (MDPI)	SJR Q2 (Comp. Netw. & Comm.)†	Continual GNN learning on the Ethereum phishing multigraph; explicitly merges parallel edges.
P3	S. Ferretti — AML GNN benchmark [29]	2025	IEEE Access (IEEE)	SJR Q1† (SJR 2024 = 0.849)	Controlled comparison of GNN families and feature sets for Bitcoin AML (Elliptic).
P4	M. Li — TFGAT [12]	2025	Scientific Reports (Springer Nature)	JCR Q1† (Multidiscipl.)	Global–local attention and pseudo-labelling for Bitcoin AML.
P5	V. K. Kasula — AJKGS-ABCM [11]	2025	Discover Artificial Intelligence (Springer Nature)	SJR Q1† (SJR 2024 = 0.876)	Multi-class Ethereum address classification with attention and jumping knowledge; reports latency.
P6	Y. Jia — LMAE4Eth [10]	2025	IEEE Trans. Inf. Forensics Security (IEEE)	SJR Q1† (Comp. Netw. & Comm.)	Transaction-semantics language model + masked graph autoencoder for Ethereum fraud; three real datasets.
P7	Z. Lin — ChronoWave-GNN [27]	2026	Scientific Reports (Springer Nature)	JCR Q1† (Multidiscipl.)	Temporal graph transformer for Bitcoin AML with latency profiling; batch-mode.
P8	S. Al-E'mari — Explainable GNNs [8]	2026	Future Internet (MDPI)	SJR Q2 (Comp. Netw. & Comm.)†	Multi-class Ethereum fraud with statistical testing and quantitative explanations.
P9	R. Rico — TAML [9]	2026	Proc. IJCAI-26 (IJCAI)	CORE2023 A*	Joint AML classification and sub-network discovery; evaluated on IBM AML, SAML-D, Elliptic++.

†Quartile taken from a publisher bibliometrics page or a secondary SJR/JCR listing (the SCImago portal blocked automated access); see Appendix B.

3.5 Screened but Excluded Candidates

Transparency about exclusions is part of a defensible review. Table 3 lists candidates that were screened and *not* counted, with reasons. Three of them appeared in our earlier Phase 2 draft and were removed from the quota after verification.

4 Detailed Individual Paper Analyses

Each of the nine included papers is analysed under the fifteen headings (A–O) used throughout this review. All numbers below were checked against the full text examined (Appendix B). Statements under headings L–O are our own assessment unless they are explicitly attributed to the authors. Details that the paper does not report are stated as such rather than inferred.

4.1 Paper 1: GeneralDyG — A Generalizable Anomaly Detection Method in Dynamic Graphs

A. Bibliographic Information and Eligibility

- **Full Title:** A Generalizable Anomaly Detection Method in Dynamic Graphs

Table 3: Screened Candidates Excluded from the Phase 2 Quota

Candidate	Venue / year	Reason for exclusion (and how it is used instead)
Chen et al., MDST-GNN [30]	Adv. Intell. Syst., 2025	Metadata verified, but the full text could not be accessed for verification (publisher bot-protection page); only the abstract could be checked. Excluded so that no unverifiable numbers enter the review.
Zhou & Abbas, unsupervised dynamic GNN [31]	IEEE Access, 2025	Evaluated only on non-financial graphs (communication and protein networks) for link prediction; insufficient relevance.
Asiri & Somasundaram, GCN fraud detection [32]	Sci. Rep., 2025	Full text (PubMed Central) contains integrity anomalies: metric definitions refer to "skin cancer", the training table reports 0.9856 training accuracy against 0.168 validation accuracy, the text calls an AUC of 0.9444 "perfect", and it states that "random feature vectors are assigned to generate synthetic features". Cited only as a cautionary example in Section 6.
Li et al., TGDAMFormer (Appl. Intell., 2026)	Applied Intelligence, 2026	Directly relevant (Elliptic++), but subscription-only; the method could not be verified beyond the abstract.
Egressy et al. [1]; Ding et al. [13]; Blanuša et al. [15]; Lin et al. [33]	AAAI-24; CIKM-24; ICAIF-24; ICAIF-24	Highly relevant but published in 2024 (outside the window); analysed as closest prior work in Section 5.
Bilgi & Atasu, MEGA-GNN [14]	TMLR (accepted, 2026)	Highly relevant, but a Q1/Q2 ranking for TMLR could not be verified; analysed as closest prior work.
TeMP-TraG [34]; Amatriciana [35]	arXiv preprints	Not verifiable as peer-reviewed 2025–2026 Q1/Q2 publications; used as related work only.

- **Authors:** Xiao Yang, Xuejiao Zhao, and Zhiqi Shen
- **Publication Year:** 2025 (published April 11, 2025)
- **Venue:** *Proceedings of the AAAI Conference on Artificial Intelligence* (AAAI-25), Vol. 39, No. 20, pp. 22001–22009
- **Publisher:** Association for the Advancement of Artificial Intelligence (AAAI)
- **DOI / Source URL:** 10.1609/aaai.v39i20.35508; open access: https://ojs.aaai.org/index.php/AAAI/article/view/35508
- **Ranking / Quartile:** CORE2023 **A*** (verified on the ICORE conference portal).
- **Relevance to This Project:** Event-level anomaly detection on continuous-time dynamic graphs, evaluated on two Bitcoin trading trust networks. Relevant to how temporal neighbourhoods around an event can be sampled; it does not address AML labels or transaction multigraphs directly.
- **Eligibility Decision: Eligible.** Original research published in 2025 at a CORE A* venue; full text examined (authors' version, arXiv:2412.16447).

B. Research Problem and Motivation

The authors argue that existing dynamic-graph anomaly detectors generalise poorly across domains because of three challenges: *data diversity* (graphs differ in structure and feature semantics), *dynamic feature capture* (structural and temporal changes are entangled), and *computational cost*. They aim for one detector that works for both node-level and edge-level anomalies across different dynamic graphs.

C. Research Objectives and Contributions

1. *Temporal ego-graph sampling:* for each event, a temporal k-hop ego-graph is extracted around it, with hierarchical tokens marking the hop levels of the ego-graph.

2. *TensGNN:* a GNN extractor that alternates node-feature and edge-feature propagation layers (the final layer is a node layer for node-level tasks and an edge layer for edge-level tasks), embedding nodes, edges and topology into the feature space.

3. *Temporal-aware Transformer:* the topological structure information of the ego-graph is overlaid on the Query and Key, while the original event features are retained.

D. Related Work and Theoretical Foundation

The paper builds on continuous-time dynamic graph (CTDG) learning (e.g., TGAT [36], TGN [37]) and on dynamic-graph anomaly detectors such as TADDY, StrGNN, SAD and SLADE, and node-level detectors such as GDN and FuSAGNet. It positions ego-graph sampling as a way to keep computation local while still capturing temporal context.

E. Dataset and Data Collection

- **Edge level:** Bitcoin-Alpha and Bitcoin-OTC, trust networks of Bitcoin users trading on the btc-alpha.com and bitcoin-otc.com platforms. Following the TADDY protocol, anomalies are injected into the test set at 1%, 5% and 10%.

- **Node level:** SWaT and WADI (water-treatment sensor graphs), used as in GDN.

F. Preprocessing and Feature Engineering

Each event is represented by its temporal k-hop ego-graph; node, edge and topological structure information are mapped into the feature space, and hierarchical tokens encode the hop level of each element. The paper does not use transaction amounts or AML-specific features.

G. Model Architecture and Technical Methodology

The ego-graph sequence is encoded by TensGNN (alternating node and edge propagation) and then by a Transformer whose attention is conditioned on the ego-graph topology (structure injected into Query and Key; original features kept in the Value path). Anomaly scores are produced for the target edge or node.

H. Experimental Setup

GeneralDyG is compared with 20 baselines spanning graph embeddings (node2vec, DeepWalk), dynamic GNNs (TGAT, TGN), dynamic-graph anomaly detectors (AddGraph, StrGNN, TADDY, SAD, SLADE) and node-level detectors (GDN, BTAD, MST-GAT, FuSAGNet, MTAD-GAT and others). Implementation hyperparameters and hardware are not reported in the version examined and are therefore not stated here.

I. Evaluation Metrics

Edge level: ROC-AUC and Average Precision (AP). Node level: F1.

J. Results and Quantitative Interpretation

- **Bitcoin-Alpha (AUC / AP, %):** 94.01 / 24.00 (1% anomalies); 95.41 / 24.02 (5%); 96.28 / 26.73 (10%).
- **Bitcoin-OTC (AUC / AP, %):** 94.66 / 27.89 (1%); 94.86 / 29.97 (5%); 95.59 / 27.13 (10%).
- **Node level (F1, %):** SWaT 85.19 (second best: FuSAGNet 83.69); WADI 60.43 (slightly below FuSAGNet, 60.70).

Note that AP remains below 0.30 even when AUC exceeds 0.94, a reminder that ROC-AUC alone overstates performance on rare anomalies [38].

K. Ablation Studies and Component Analysis

Bitcoin-Alpha (10%) AUC / AP and WADI F1: full model 96.28 / 26.73 and 60.43; without ego-graph sampling 96.01 / 19.33 and 59.45; without TensGNN 92.02 / 22.63 and 55.13; without the Transformer 93.71 / 20.20 and 58.46. Ego-graph sampling contributes most to AP; TensGNN contributes most to AUC and WADI F1.

L. Limitations and Weaknesses

1. *Injected anomalies:* the edge-level evaluation uses synthetically injected anomalies, not labelled laundering or fraud.
2. *No transaction semantics:* the datasets carry no transfer amounts, and repeated interactions between the same pair are not modelled as an explicit multigraph.
3. *No systems evaluation:* no per-event latency, throughput or streaming evaluation is reported.

M. Future Research Directions and Threats to Validity

The authors propose studying the interpretability of anomaly detection in dynamic graphs and providing stronger theoretical support. The main threat to validity for our purposes is construct validity: injected edge anomalies may not resemble coordinated laundering patterns.

N. Relevance to Our Proposed Research

GeneralDyG shows that sampling a bounded temporal neighbourhood around each event is an effective and computationally contained way to obtain temporal context. This supports our bounded, view-restricted sampler (Step 5).

O. Paper-Specific Conclusion and Comparative Positioning

GeneralDyG is a strong, generalisable dynamic-graph anomaly detector, but its evaluation does not involve labelled financial crime, transaction amounts, parallel transfers or streaming latency, which are the dimensions our study targets.

4.2 Paper 2: Tian & Zhang — Continual Graph Learning with Knowledge-Augmented Replay

A. Bibliographic Information and Eligibility

- **Full Title:** Continual Graph Learning with Knowledge-Augmented Replay: A Case for Ethereum Phishing Detection

- **Authors:** Zonggui Tian and Du Zhang
- **Publication Year:** 2025 (published August 22, 2025)
- **Venue:** *Electronics* (MDPI), Vol. 14, No. 17, Art. no. 3345
- **Publisher:** MDPI
- **DOI / Source URL:** 10.3390/electronics14173345
- **Ranking / Quartile:** SJR **Q2**, Computer Networks and Communications (secondary listing; see Appendix B).
- **Relevance to This Project:** Continual learning on the XBlock Ethereum phishing network, our conditional real-data extension; explicitly reports collapsing parallel transfers.
- **Eligibility Decision: Eligible.** Peer-reviewed 2025 article in a Q2 venue; full text examined (MDPI open access).

B. Research Problem and Motivation

GNNs retrained on new data forget earlier patterns (catastrophic forgetting), while retraining on the full history is expensive. The authors study *sample-incremental* continual graph learning on the evolving Ethereum transaction network and ask how knowledge from later stages can reinforce, rather than overwrite, earlier knowledge.

C. Research Objectives and Contributions

1. *Temporal partitioning:* the network is split chronologically into 10 incremental transaction networks (ITNs) with an approximately equal number of phishing accounts each.
2. *Evolutionary transaction subgraphs:* incremental (ITSG) and accumulative (ATSG) transaction subgraphs describe how an account's transactions evolve; detection is reformulated as classification of first-order subgraphs.
3. *Knowledge-Augmented Replay (KAR):* a memory buffer stores phishing accounts that reappear in later stages (plus an equal number of reappearing non-phishing accounts) and replays their accumulated subgraphs with inter-stage edges retained.
4. *An ϵ-k bounded convergence criterion* for continual learning, linking average accuracy to backward transfer.

D. Related Work and Theoretical Foundation

The work draws on continual learning (regularisation methods such as EWC and the topology-aware TWP; distillation such as LwF; and experience replay) and on Ethereum phishing detection with graph embeddings and GNNs. Its key argument is that *inter-stage edges* (connections between the current stage and earlier stages) carry knowledge that most continual-graph-learning methods ignore.

E. Dataset and Data Collection

- **XBlock Ethereum transaction network:** 2,973,489 addresses, 13,551,303 transactions, 1,165 labelled phishing addresses; the largest connected component (2,973,382 addresses; 13,551,214 transactions; 1,157 phishing) is used.
- **Partitioning:** the 1,157 phishing accounts are ordered by first timestamp and allocated to 10 ITNs (116 phishing accounts in each of the first seven, 115 in each of the last three).

- **Class balance:** each stage dataset contains all phishing subgraphs and an *equal number* of randomly selected non-phishing subgraphs.

F. Preprocessing and Feature Engineering

- **Parallel-edge collapse:** each snapshot "is a multigraph"; the authors simplify it "into a simple graph by merging parallel edges and summing their transaction amounts".
- **Node features:** number of first-order neighbours, in-degree, out-degree, total degree, in-strength, out-strength and total strength. The paper also lists node labels as an available feature when the task is reformulated as graph classification; any reuse of this design must ensure that the label of the account being classified is never an input.

G. Model Architecture and Technical Methodology

Four backbones (GCN, GAT, GATv2, GIN) share a template of two graph convolution layers (50% dropout), global pooling to a subgraph vector and a linear classifier. KAR, naïve fine-tuning and full retraining differ in how the central-node set and subgraphs are constructed at each stage.

H. Experimental Setup

Within each stage, the balanced dataset is randomly shuffled and split 70/30 into training and test sets. Hidden size $d = 32$; maximum epochs $\{100, 300, 500\}$; learning rate $\{0.001, 0.003, 0.005\}$; batch size 32; PyTorch 2.1.2 and PyTorch Geometric on an NVIDIA V100 (16 GB). EWC, LwF and TWP are compared under a unified protocol.

I. Evaluation Metrics

Average accuracy (ACC) over all stages and backward transfer (BWT), using the standard continual-learning definitions cited by the authors.

J. Results and Quantitative Interpretation

- With inter-stage edges retained, KAR and retraining reach or exceed 90% stage-wise average accuracy (GIN-based variants remain stable or decline slightly); KAR matches retraining while using less memory.
- With inter-stage edges removed, naïve fine-tuning and replay stabilise around 50% and retraining around 60%, with negative backward transfer indicating forgetting.
- With inter-stage edges retained, KAR also outperforms EWC, LwF and TWP on both accuracy and forgetting.

K. Ablation Studies and Component Analysis

The central ablation (with versus without inter-stage edges) shows that cross-stage connectivity, not only parameter regularisation, determines whether forgetting occurs on this network.

L. Limitations and Weaknesses

1. *Multigraph collapse:* summing parallel transfers removes per-transfer amounts, timing and burst counts.
2. *Balanced evaluation:* 1:1 class balance and accuracy as the main metric do not reflect precision at the real prevalence (1,165/2,973,489 $\approx 0.04\%$ of addresses).

3. *Random splits within stages:* the chronological stage structure is respected across stages, but not within each stage.
4. *First-order subgraphs only.*

M. Future Research Directions and Threats to Validity

The authors propose adapting knowledge augmentation to other continual-learning settings, refining the ϵ-bounded convergence criterion, and applying KAR to broader cryptocurrency anomaly detection, including AML. Threats to validity for our use: the balanced accuracy metric and the label-as-feature design described in heading F.

N. Relevance to Our Proposed Research

KAR is direct evidence for Gap G1 (explicit summing of parallel transfers on the same Ethereum network we use) and shows that temporal connectivity across time periods matters. This motivates retaining state across time rather than treating periods independently, which is why the gated extension considers node memory.

O. Paper-Specific Conclusion and Comparative Positioning

KAR demonstrates that preserving cross-period connectivity prevents forgetting on Ethereum, but its simple-graph representation, balanced datasets and stage-level updates leave per-transfer modelling and decision timing untested.

4.3 Paper 3: Ferretti et al. — Enhancing AML Frameworks in Cryptocurrency Transactions

A. Bibliographic Information and Eligibility

- **Full Title:** Enhancing Anti-Money Laundering Frameworks: An Application of Graph Neural Networks in Cryptocurrency Transaction Classification
- **Authors:** Stefano Ferretti, Gabriele D'Angelo, and Vittorio Ghini
- **Publication Year:** 2025 (published March 17, 2025)
- **Venue:** *IEEE Access*, Vol. 13, pp. 50201–50215
- **Publisher:** IEEE
- **DOI / Source URL:** 10.1109/ACCESS.2025.3552240
- **Code:** https://github.com/stefano-ferretti/aml-elliptic-gnn
- **Ranking / Quartile:** SJR **Q1** (secondary listing); SJR 2024 = 0.849 (IEEE Access bibliometrics page).
- **Relevance to This Project:** Controlled comparison of GNN families and feature sets for Bitcoin AML on Elliptic; isolates the contribution of features versus graph structure.
- **Eligibility Decision: Eligible.** Peer-reviewed 2025 Q1 journal article; full text examined (IEEE Xplore, open access).

B. Research Problem and Motivation

Practitioners lack controlled guidance on which GNN architectures and feature sets work best for classifying illicit cryptocurrency transactions. The authors analyse the Elliptic dataset (class imbalance, correlated features) and compare GNN variants under several feature configurations.

C. Research Objectives and Contributions

1. Comparison of GCN, GAT, GATv2, GraphSAGE and Chebyshev convolutions.
2. Three architectural variants per family: a two-layer baseline, a variant with a *final* linear layer (+Lin), and a variant with the linear layer and skip connections (+Lin+Skip).
3. Three feature configurations: transaction features only (`tx`), all features (`tx+agg`), and transaction features plus a reduced set of aggregated features excluding those with very high correlation (`tx+agg_red`).

D. Related Work and Theoretical Foundation

Builds on spatial and spectral GNNs [4–6] and on the Elliptic benchmark [39]. Skip connections are motivated as a remedy for over-smoothing in message passing.

E. Dataset and Data Collection

Elliptic: 203,769 transaction nodes and 234,355 directed edges; each node has 167 features, of which the first 94 relate to the transaction itself (e.g., numbers of inputs and outputs) and the rest are aggregated from neighbouring transactions. Timestamps are anonymised time steps (relative ordering only).

F. Preprocessing and Feature Engineering

Feature correlation analysis reveals many aggregated-feature pairs with correlation above 0.8, and several above 0.9; `tx+agg_red` excludes aggregated features with "extraordinary" correlation to other features. The number of excluded features is not needed for our argument and is not repeated here.

G. Model Architecture and Technical Methodology

Each model has two graph-convolution layers with ReLU and dropout; +Lin adds a final linear layer; +Lin+Skip adds skip connections that preserve information from earlier message-passing rounds before the final linear layer and log-softmax output.

H. Experimental Setup

Adam with an initial learning rate of 9×10^{-3}, reduced by a factor of 0.9 when validation loss does not improve for 100 epochs; 64 hidden units (`tx`) or 110 (`tx+agg`, `tx+agg_red`); up to 13,000 epochs with early-stopping patience of 1,000; 15 repetitions per configuration; one NVIDIA L40 (48 GB) node. The train/validation/test split protocol is not described in the text examined.

I. Evaluation Metrics

Precision, recall and F1 for the illicit class; an averaged F1 over both classes (described as micro-averaged in the methods and reported as "M.A. F1" in the results); training time.

J. Results and Quantitative Interpretation

- Best configuration: Chebyshev+Lin on `tx+agg_red`, with illicit F1 = **0.906**, precision 0.955, recall 0.863, and averaged F1 0.983.
- GATv2 reaches illicit F1 above 0.885 in configurations with aggregated features; GraphSAGE on `tx+agg` has the highest recall (0.818); GCN-based models perform relatively worse.

- `tx+agg` generally outperforms `tx`; `tx+agg_red` gives "only marginal improvements" but the best overall score.
- Removing the dataset features and keeping only graph structure (plus structural node metrics) produced "very poor results".

K. Ablation Studies and Component Analysis

The feature-configuration and architecture-variant grid acts as an ablation: aggregated neighbourhood features help; the final linear layer and skip connections generally help, but not uniformly across architectures (largest gains for GAT).

L. Limitations and Weaknesses

1. *Undocumented split protocol*, so leakage across time steps cannot be ruled out from the text.
2. *Transaction graph without amounts on edges*: Elliptic has no parallel transfers or per-edge values, so multigraph effects cannot be studied.
3. *Offline evaluation* only; no streaming or latency analysis.

M. Future Research Directions and Threats to Validity

The authors call for more transparent datasets, potentially synthetic transaction networks generated by agent-based models, further study of heterogeneous GNNs (their preliminary heterogeneous results were lower), and methods to explain which nodes, edges and features drive predictions.

N. Relevance to Our Proposed Research

(1) The structure-only failure supports keeping transaction attributes on every event (Step 4).
(2) Chebyshev+Lin (0.906) is a reported reference point for Elliptic, not a baseline we reproduce.
(3) The authors' call for synthetic agent-based data supports using the IBM simulator data [3], alongside an independent simulator and a real chain as extensions.

O. Paper-Specific Conclusion and Comparative Positioning

A careful offline benchmark showing that features dominate and that architectural details matter; it does not address multigraph structure, streaming, or a documented temporal protocol.

4.4 Paper 4: TFGAT — Global-Local Graph Attention with Cyclic Pseudo-Labels

A. Bibliographic Information and Eligibility

- **Full Title:** Global-local graph attention with cyclic pseudo-labels for bitcoin anti-money laundering detection
- **Authors:** Meng Li, Lu Jia, and XinQiao Su
- **Publication Year:** 2025 (published July 2, 2025)
- **Venue:** *Scientific Reports* (Springer Nature), Vol. 15, Art. no. 22668
- **Publisher:** Springer Nature
- **DOI / Source URL:** 10.1038/s41598-025-08365-9
- **Ranking / Quartile:** JCR **Q1**, Multidisciplinary Sciences (secondary listing).

- **Relevance to This Project:** Combines global (Transformer) and local (GAT) attention with semi-supervised pseudo-labelling for Bitcoin AML; reports parameters and training times.
- **Eligibility Decision: Eligible.** Peer-reviewed 2025 Q1 article; full text examined (open access).

B. Research Problem and Motivation

The authors identify three challenges: GCNs assign uniform neighbour weights and can be misled by concealed illicit transactions; node-level GCNs fail to extract global information; and labels are expensive, so most transactions are unlabelled (157,205 of Elliptic's 203,769 nodes, about 77%).

C. Research Objectives and Contributions

1. **TFGAT** with a Global–Local Attention Mechanism: a Transformer encoder extracts global information across transactions, and a GAT layer then attends over local neighbours.
2. **DCPLU** (Deep Cyclic Pseudo-Label Updating): a supervised TFGAT generates pseudo-labels for unlabelled nodes; semi-supervised training on true plus pseudo-labels then updates the pseudo-labels cyclically until validation performance converges, without manifold or Euclidean-distance assumptions.

D. Related Work and Theoretical Foundation

Positions itself against supervised GCN-based AML models (GCN, Skip-GCN, EvolveGCN, temporal attention models) and semi-supervised label-propagation methods, whose similarity assumptions the authors argue do not hold for concealed illicit transactions.

E. Dataset and Data Collection

Elliptic: over 200,000 transaction nodes, about 234,000 edges, 49 time steps (about two weeks apart), labels illicit/licit/unknown; "AF" denotes all 166 features and "LF" the 94 local features in the baseline table.

F. Preprocessing and Feature Engineering

The paper uses the Elliptic features directly; no additional feature engineering or normalisation procedure is described in the text examined. Class imbalance is handled by a weighted cross-entropy loss (weight 1.0 for the legal class; illegal-class weight 10 in the supervised stage and 6 in the semi-supervised stage, chosen by Bayesian optimisation over $[2, 15]$).

G. Model Architecture and Technical Methodology

Two TransformerEncoder layers with 4 heads and $d_{\text{model}} = 512$ (selected by Bayesian optimisation from 1–4 layers, 2–8 heads and $\{192, 256, 512, 1024\}$), followed by multi-head GAT attention over neighbours and a two-layer classifier. DCPLU retrains the same architecture with the label set y_{new} (true labels plus current pseudo-labels), so the parameter count, and hence inference cost, is unchanged.

H. Experimental Setup

1,000 epochs with Adam; L_2 weight 5×10^{-5}; learning rate reduced by 20% (supervised) or 10% (semi-supervised) every 100 iterations; model selection by validation F1. Twenty-one baselines are listed, with results taken from earlier publications (Weber et al., Vassallo et al., Lo et al.) rather than re-run. The train/validation/test split is not described in the text examined. The printed F1 formula (Eq. 20) omits the factor 2.

I. Evaluation Metrics

Illicit-class precision, recall and F1; ROC-AUC.

J. Results and Quantitative Interpretation

- TFGAT: precision 0.9813, recall 0.8322, F1 0.9006.
- TFGAT + DCPLU: precision **0.9814**, recall **0.8390**, F1 **0.9046**; AUC 0.9187.
- Classical label-propagation variants perform markedly worse, which the authors attribute to violated smoothness assumptions.

K. Ablation Studies and Component Analysis

Parameters and training time: Transformer only 6.8M, 4.5 h (the text states 4.7 h); GAT only 6.6M, 3.2 h; TFGAT 6.7M, 4.1 h; TFGAT + DCPLU 6.7M, 5.1 h. F1: Transformer 0.8603 (0.8674 with DCPLU); GAT 0.8062 (0.8154); TFGAT 0.9006 (0.9046). Removing the Transformer causes the largest drop.

L. Limitations and Weaknesses

1. *Undocumented split* and *borrowed baselines* prevent a controlled comparison.
2. *Global attention over all transactions* scales quadratically with the number of nodes attended to; with 4–5 h training it is unsuited to frequent updates.
3. *Static graph*: time steps are not used to model temporal dynamics.

M. Future Research Directions and Threats to Validity

The paper does not state explicit future-work directions in the version examined. The main threats to validity are the unreported split and the non-reproduced baselines.

N. Relevance to Our Proposed Research

TFGAT shows that wider context improves recall, but at a cost that is incompatible with streaming. This motivates bounding the temporal neighbourhood per event (Step 5) and measuring cost explicitly (Step 8).

O. Paper-Specific Conclusion and Comparative Positioning

High illicit-class F1 on Elliptic with a static, global-attention model; the evaluation protocol and cost leave deployability under streaming unaddressed.

4.5 Paper 5: Kasula et al. — Blockchain Address Classification Using AJK-GraphSAGE

A. Bibliographic Information and Eligibility

- **Full Title:** Blockchain address classification using graph neural networks for enhanced security and analytics
- **Authors:** Vinay Kumar Kasula, Akhila Reddy Yadulla, Mounica Yenugula, Bhargavi Konda, and Supraja Ayyamgari
- **Publication Year:** 2025 (published October 24, 2025)
- **Venue:** *Discover Artificial Intelligence* (Springer Nature), Vol. 5, Art. no. 285
- **Publisher:** Springer Nature
- **DOI / Source URL:** 10.1007/s44163-025-00508-1
- **Ranking / Quartile:** SJR **Q1** (SJR 2024 = 0.876; secondary listing).
- **Relevance to This Project:** Address-level Ethereum classification with attention over layer depths; one of only two reviewed studies that report per-batch inference time.
- **Eligibility Decision: Eligible.** Original 2025 research article in a Q1 journal; full text examined (open access).

B. Research Problem and Motivation

Manual-feature classifiers generalise poorly across platforms, and plain neighbourhood aggregation treats all neighbours and all depths equally. The authors aim to classify Ethereum addresses into functional categories (useful for forensics and AML) using adaptive multi-depth graph representations.

C. Research Objectives and Contributions

1. **AJKGS-ABCM:** an attention-guided jumping-knowledge GraphSAGE for address classification under an account-balance model, with attention over neighbours and over layer depths.
2. A labelled 11-class Ethereum address dataset (22,312 addresses).
3. Training- and inference-time profiling against GraphSAGE, GATv2 and a Graph Transformer.

D. Related Work and Theoretical Foundation

Builds on GraphSAGE [5] and Jumping Knowledge networks [40]; baselines include KNN on handcrafted features, DeepWalk, Node2Vec, Struc2Vec, I2BGNN, GCN and GraphSAGE.

E. Dataset and Data Collection

- **Labels:** 22,312 addresses compiled from Etherscan, a Kaggle labelled dataset, XLabelCloud and manually validated social-media sources, in 11 classes: Exchange, Mining pool/Mining, Gambling, Token contract, Darknet, Wallet, Scam, ICO (fundraising), Hacker, Mixer and General user. The authors describe the imbalance as moderate (Exchange and General user frequent; Mixer and Darknet rare).
- **Transactions:** 3,437,967 transactions directly involving the labelled addresses, extended with neighbouring interactions to 58,172,811; after removing failed transactions, 59,465,095 valid transactions are used to build node features and the graph.

F. Preprocessing and Feature Engineering

Transaction data were collected via the Etherscan API and the Google BigQuery Ethereum dataset; an adjacency structure and account-state node features are built from the transactions. Mini-batches are drawn with stratified sampling to mitigate class imbalance. The exact feature list and normalisation are not restated here.

G. Model Architecture and Technical Methodology

K GraphSAGE layers with neighbour attention produce layer outputs $\mathbf{h}_i^{(1)}, \ldots, \mathbf{h}_i^{(K)}$. Layer-level attention weights are computed as

$$\mathbf{A}_i = \mathrm{softmax}\Big(\mathbf{W}_a \cdot \mathrm{CONCAT}\left(\mathbf{h}_i^{(1)}, \ldots, \mathbf{h}_i^{(K)}\right)\Big), \tag{1}$$

the layer outputs are fused with these weights (jumping knowledge), and a fully connected softmax layer outputs the class probabilities.

H. Experimental Setup

Training ratios of 60%, 70% and 80% with random splits, 20 repetitions each; the 80:20 split is used for the main comparison; embedding dimension 128; timing on an NVIDIA RTX A6000 (48 GB).

I. Evaluation Metrics

Precision, recall, micro-F1 and macro-F1; training time per epoch and inference time per batch.

J. Results and Quantitative Interpretation

- 80:20 split: AJKGS-ABCM precision 0.862, recall 0.852, micro-F1 0.849, macro-F1 **0.856**; GraphSAGE 0.823, I2BGNN 0.824, GCN 0.807 and KNN 0.648 (macro-F1).
- Cost: 14.9 s per epoch and 20.8 ms per batch, versus 12.3 s and 18.4 ms for GraphSAGE (GATv2: 16.7 s and 24.6 ms; Graph Transformer: 21.5 s and 32.1 ms).

K. Ablation Studies and Component Analysis

The comparison with plain GraphSAGE isolates the attention and jumping-knowledge additions: +3.3 percentage points macro-F1 (0.856 vs. 0.823) for about 21% more training time per epoch and +2.4 ms per batch.

L. Limitations and Weaknesses

1. *Random splits* over a time-evolving graph allow information from later transactions to inform earlier labels.
2. *Static, cumulative graph*: timing and multiplicity of transactions are not modelled.
3. *Per-batch timing only*: no arrival-driven latency, throughput or freshness analysis.

M. Future Research Directions and Threats to Validity

The conclusion does not state explicit future-work directions in the version examined. The main threat to validity is the random split protocol.

N. Relevance to Our Proposed Research

Kasula et al. provide one of the few cost measurements in the reviewed set and show that attention-based aggregation adds modest overhead. We do not adopt jumping knowledge. Their per-batch timing motivates our more complete systems metrics (Step 8).

O. Paper-Specific Conclusion and Comparative Positioning

A solid multi-class address classifier with useful cost reporting; its random splits and static graph leave temporal validity and streaming behaviour open.

4.6 Paper 6: LMAE4Eth — Ethereum Fraud Detection with Transaction Semantics and Masked Graph Embedding

A. Bibliographic Information and Eligibility

- **Full Title:** LMAE4Eth: Generalizable and Robust Ethereum Fraud Detection by Exploring Transaction Semantics and Masked Graph Embedding
- **Authors:** Yifan Jia, Yanbin Wang, Jianguo Sun, Ye Tian, and Peng Qian
- **Publication Year:** 2025
- **Venue:** *IEEE Transactions on Information Forensics and Security* (TIFS), Vol. 20, pp. 10260–10274
- **Publisher:** IEEE
- **DOI / Source URL:** 10.1109/TIFS.2025.3612149; authors' version: arXiv:2509.03939v1
- **Ranking / Quartile:** SJR **Q1** in Computer Networks and Communications and in Safety, Risk, Reliability and Quality (secondary listings).
- **Relevance to This Project:** Ethereum fraud detection on three real datasets, including the XBlock network, our conditional real-data extension.
- **Eligibility Decision: Eligible.** Peer-reviewed 2025 Q1 journal article. Metadata verified on Crossref; technical details below were checked against the authors' version (arXiv:2509.03939v1) and may differ in minor ways from the version of record.

B. Research Problem and Motivation

The authors argue that (i) existing detectors treat transaction histories as numerical sequences and miss their contextual meaning; (ii) Ethereum transaction records are highly homogeneous, which makes discriminative account embeddings hard to learn; and (iii) self-supervised graph methods that rely on structure reconstruction perform poorly on node-level fraud detection and scale poorly.

C. Research Objectives and Contributions

1. **TxCLM**, a transaction-token contrastive language model that turns numerical transaction records into text-like sequences and is pretrained with masked-transaction and token-aware contrastive objectives.
2. **MAGAE**, a masked account-graph autoencoder that reconstructs masked account node features, combined with layer-neighbour (LABOR) sampling for large graphs.
3. A **cross-attention fusion network** that combines the language-model and graph embeddings, with expert account features.

D. Related Work and Theoretical Foundation

Positions itself against graph-embedding methods (DeepWalk, Role2Vec, Trans2Vec), transaction-sequence Transformers (BERT4ETH, ZipZap, T5, Longformer), GNNs and graph-level methods (GCN, GAT, GraphSAGE, GRAND, TSGN, GrabPhisher, CATALOG and others) and graph autoencoders (GAE, VGAE, GATE).

E. Dataset and Data Collection

1. **MulDiGraph** (XBlock): 2,973,489 accounts, 13,551,303 transactions, 1,165 phishing accounts.
2. **B4E**: 597,258 accounts (3,220 phishing), collected from an Ethereum node for January 2017 to May 2022.
3. **SPN**: 496,740 accounts (5,619 phishing) and 831,082 transaction edges, collected up to June 2024.

F. Preprocessing and Feature Engineering

Each transaction becomes a short text of the form "amount: v, direction: d, timestamp: τ" ($d \in \{-1, 1\}$); an account's transactions form one "sentence", and addresses are excluded from the text. In the account graph, edge weights are proportional to the number of transactions between two accounts. The main experiments use a random 7:1:2 node split; an additional experiment splits by connected component so that no edges cross sets.

G. Model Architecture and Technical Methodology

TxCLM (a BERT-style encoder trained with masked-transaction and token-aware contrastive losses) and MAGAE (a masked graph autoencoder with a scaled-cosine reconstruction error) produce account embeddings. A cross-attention fusion network with learnable aggregation and fusion tokens combines them for classification with a cross-entropy loss.

H. Experimental Setup

Comparison against a large set of baselines (the paper reports 21 in the experiments section and 15 in the abstract) on the three datasets under the random 7:1:2 split. Wall-clock training time and hardware are not reported in the version examined.

I. Evaluation Metrics

Precision, recall, F1 and balanced accuracy.

J. Results and Quantitative Interpretation

- MulDiGraph: F1 **0.8960** (best baseline T5: 0.7936).
- B4E: F1 **0.8143** (best baseline CATALOG: 0.7258).
- SPN: F1 **0.8002** (best baseline GrabPhisher: 0.7353).
- With 1% of B4E labels, F1 is 56.28%; under the connected-component split, F1 is 0.8996, 0.8216 and 0.7895 on the three datasets (D1–D3 in the paper).

K. Ablation Studies and Component Analysis

On B4E: full model 0.8143; without the language model 0.6446 (−16.97 points); without the graph model 0.7632 (−5.11 points); MAGAE replaced by a plain GAE 0.7785; fusion replaced by addition 0.7625. Transaction semantics contribute more than the graph branch.

L. Limitations and Weaknesses

1. *Batch mode:* the method assumes a graph snapshot and sufficient transaction history; incremental updates are not supported (stated by the authors).
2. *Random node split* in the main experiments, with no temporal split.
3. *Count-weighted edges:* individual transfers enter only through the text sequences, not as separate edges in message passing.

M. Future Research Directions and Threats to Validity

The authors plan streaming and incremental learning and collaboration with blockchain data providers for more diverse, up-to-date and multi-chain data. Threats to validity: random splits, and the use of public datasets that may not reflect evolving behaviour (acknowledged by the authors).

N. Relevance to Our Proposed Research

LMAE4Eth is direct evidence for G2 (batch-only; streaming named as future work) and G3 (random splits). Its ablation shows that per-transaction information (amount, direction, timing) is highly informative, which supports keeping every transfer as a separate event with its attributes (Step 4).

O. Paper-Specific Conclusion and Comparative Positioning

State-of-the-art offline Ethereum fraud detection across three real datasets; its batch design and random splits leave temporal deployment questions open.

4.7 Paper 7: ChronoWave-GNN — A Wavelet-Temporal Graph Transformer for Bitcoin AML

A. Bibliographic Information and Eligibility

- **Full Title:** Detecting illicit transactions in bitcoin: a wavelet-temporal graph transformer approach for anti-money laundering
- **Authors:** Ziqian Lin, Qining Luo, Dongze Wu, Jie Shen, Lingying Li, Xunyi Nong, and Zhenkai Qin
- **Publication Year:** 2026 (published January 13, 2026)
- **Venue:** *Scientific Reports* (Springer Nature), Vol. 16, Art. no. 1548
- **Publisher:** Springer Nature
- **DOI / Source URL:** 10.1038/s41598-025-23901-3
- **Ranking / Quartile:** JCR **Q1**, Multidisciplinary Sciences (secondary listing).

- **Relevance to This Project:** Temporal graph transformer for Bitcoin AML with inference-latency profiling and a sliding-window update prototype; the authors explicitly name streaming and temporal memory as open problems.
- **Eligibility Decision: Eligible.** Peer-reviewed 2026 Q1 article; full text examined (open access).

B. Research Problem and Motivation

The authors argue that graph-based AML methods overlook the *frequency* characteristics of transactional behaviour: laundering mixes rapid bursts with slow, low-frequency layering, so detectors should capture non-stationary, multi-scale patterns.

C. Research Objectives and Contributions

1. **ChronoWave-GNN**, combining discrete wavelet features with temporal encodings in a temporal graph transformer (TGAT+).
2. A time–frequency view of transaction graphs as non-stationary spatio-temporal signals.
3. Experiments on robustness, temporal generalisation and inference efficiency.

D. Related Work and Theoretical Foundation

Builds on TGAT-style temporal attention [36], graph wavelet methods and AML-specific temporal models; it compares itself with Amatriciana and with spatio-temporal GNNs.

E. Dataset and Data Collection

Main benchmark: Elliptic (transactions as nodes, 166 features, 49 time steps); unknown-label nodes are removed before graph construction. The paper states that 45,576 transactions are labelled, about 21% of them illicit; the canonical Elliptic release has 46,564 labelled transactions of which 4,545 (about 9.8%) are illicit. Cross-dataset results are also reported on IBM AMLSim and an Ethereum phishing graph of nearly three million nodes.

F. Preprocessing and Feature Engineering

A level-2 Haar wavelet transform is applied to each node's feature vector to obtain low-frequency approximation coefficients, which are standardised and concatenated with the raw features. Each node's time step is encoded with an 8-dimensional sinusoidal encoding followed by a learnable linear projection. The headline results use a stratified 80/10/10 split of labelled nodes; a separate experiment uses chronological rolling windows.

G. Model Architecture and Technical Methodology

TGAT+ is a 3-layer TransformerConv network with ELU activations and dropout 0.4; a linear layer with softmax produces class probabilities. Training uses label smoothing ($\epsilon = 0.1$), AdamW (learning rate 0.005, weight decay 5×10^{-4}), cosine annealing, up to 200 epochs and early stopping on validation F1 (patience 20).

H. Experimental Setup

PyTorch Geometric on one NVIDIA A100; five random seeds; baselines GAT, GraphSAGE, T-GCN, TGAT, DySAT and GraphMLP (with further comparisons in the paper's conclusion and cross-dataset tables).

I. Evaluation Metrics

Accuracy, precision, recall and F1 (averaged over five runs); inference latency per batch (mean and P95) and throughput.

J. Results and Quantitative Interpretation

- Elliptic: accuracy **0.9802** and F1 **0.9799**.
- Inference (TorchScript, single GPU): about **8.4 ms per batch**, P95 below 10 ms, throughput above 5.5 million nodes per second; an INT8 CPU variant and an "OnlineGraphStore" with sliding-window updates are also demonstrated.
- An edge-feature variant gives small but statistically significant gains (about 0.3–0.5 points).

K. Ablation Studies and Component Analysis

Removing any component degrades performance; dropout matters most for generalisation. Removing the wavelet features lowers accuracy, precision, recall and F1 by roughly 0.01 (the authors describe this as modest but consistent). Level-2 decomposition works best, and fixed sinusoidal encodings perform on par with learnable Time2Vec.

L. Limitations and Weaknesses

1. *Batch mode on snapshots:* in the authors' words, the model "operates on static graph snapshots in batch mode".
2. *Split protocol of the headline result:* a stratified random split of labelled nodes allows later time steps to inform earlier ones (the rolling-window experiment partly addresses this).
3. *Reporting inconsistency:* the stated labelled-node count and illicit share differ from the canonical Elliptic statistics, which complicates comparison with other work.
4. *Per-batch latency only:* no arrival-driven queueing or freshness analysis.

M. Future Research Directions and Threats to Validity

The authors call for dynamic GNNs, temporal memory mechanisms and low-latency streaming architectures capable of online inference, as well as learnable spectral modules and better interpretability. Threats to validity: the split protocol and the dataset statistics noted above.

N. Relevance to Our Proposed Research

ChronoWave-GNN is direct evidence for G2: even a latency-profiled, efficient model is evaluated in batch mode, and its authors name temporal memory and streaming as future work. Its per-batch latency figures are a reference point for the systems metrics of Step 8, not a comparison target (different data, task and hardware).

O. Paper-Specific Conclusion and Comparative Positioning

Efficient and accurate on Elliptic under its protocol, but snapshot-based. It leaves open how quality depends on when a decision is taken, which our work studies.

4.8 Paper 8: Al-E'mari et al. — Explainable GNNs for Multi-Class Fraudulent Address Classification

A. Bibliographic Information and Eligibility

- **Full Title:** Explainable Graph Neural Networks for Multi-Class Fraudulent Address Classification in Ethereum
- **Authors:** Salam Al-E'mari, Yousef Sanjalawe, Salam Fraihat, and Ahd Aljarf
- **Publication Year:** 2026 (published August 28, 2026)
- **Venue:** *Future Internet* (MDPI), Vol. 18, No. 9, Art. no. 461
- **Publisher:** MDPI
- **DOI / Source URL:** 10.3390/fi18090461
- **Ranking / Quartile:** SJR **Q2**, Computer Networks and Communications (secondary listing).
- **Relevance to This Project:** Multi-class Ethereum fraud detection under extreme imbalance, with paired statistical testing across seeds and quantitative explanation metrics.
- **Eligibility Decision: Eligible.** Peer-reviewed 2026 article in a Q2 venue; full text examined (MDPI open access).

B. Research Problem and Motivation

Most Ethereum fraud studies are binary and cannot distinguish fraud types; graph models are rarely explained; and the contribution of graph structure over address-level features is usually assumed rather than tested.

C. Research Objectives and Contributions

1. A multi-class version of the BLTE dataset (Normal, Phishing, Scamming).
2. A comparison of GAT, GCN and GraphSAGE with feature-only learners (MLP, Random Forest, XGBoost) and a re-implemented Chebyshev GCN over ten seeds, with exact Wilcoxon signed-rank tests, Holm correction, effect sizes and confidence intervals.
3. Quantitative GNNExplainer evaluation (fidelity, sparsity, stability), plus construct-validity checks on edge construction and label noise.

D. Related Work and Theoretical Foundation

Builds on GNN-based Ethereum fraud detection, post-hoc explanation with GNNExplainer [41], and Demšar's recommendations for classifier comparison [42].

E. Dataset and Data Collection

- 73,033 addresses with 23 behavioural features and 6,588 directed edges; Normal 72,865 (99.77%), Phishing 69 (0.09%), Scamming 99 (0.14%). A single "Fake ICO" address was removed.
- 88.03% of addresses are isolated; the largest connected component has 1,187 nodes.
- Edges are built from transaction records between addresses in the node set, and *duplicate transaction edges are removed* to obtain a clean directed edge list.

F. Preprocessing and Feature Engineering

Labels are derived by matching addresses to an external attack-category table. Splits are stratified 70/15/15 (random, per seed). The class-weighted cross-entropy uses weights inversely proportional to class frequency.

G. Model Architecture and Technical Methodology

Two-layer GAT (8 heads), GCN and GraphSAGE (mean aggregator) with 64 hidden units, ReLU and dropout 0.5, followed by a softmax classifier. GNNExplainer learns edge and feature masks for each explained node.

H. Experimental Setup

Adam (learning rate 0.01, weight decay 5×10^{-4}), 300 full-batch epochs; ten stratified seeds; PyTorch 2.0.1 and PyTorch Geometric 2.3.1 on an NVIDIA RTX 3090 (24 GB).

I. Evaluation Metrics

Accuracy, macro precision/recall/F1, weighted F1, per-class metrics and absolute-count confusion matrices; explanation fidelity$^-$ (edge, feature, joint), sparsity and stability.

J. Results and Quantitative Interpretation

- Ten-seed means: GraphSAGE macro-F1 **51.08 ± 1.54%** (accuracy 98.84%, weighted F1 99.27%); MLP 49.03 ± 2.16%; XGBoost 45.33%; Chebyshev GCN 46.25%; GCN 38.68%; GAT 24.93%.
- GraphSAGE vs. MLP is **not significant** after Holm correction (adjusted $p = 0.074$; effect size $r = 0.60$; 95% CI of the macro-F1 difference $[-0.2, 4.3]$ points); GraphSAGE is significantly better than the other models.
- Even GraphSAGE reaches only about 14% precision on Phishing and 21% on Scamming; each test split contains only 10 Phishing and 15 Scamming addresses.
- Explanations (GraphSAGE): joint fidelity$^-$ 31.88%, sparsity 90.23%, stability 71.52%.

K. Ablation Studies and Component Analysis

Class weighting raises macro recall from 35.62% to 80.75% and macro-F1 from 34.41% to 51.12%, at a cost of 0.84 points of accuracy. Removing the connectivity feature group lowers macro-F1 by 7.44 points and removing temporal features by 6.91 points.

L. Limitations and Weaknesses

1. *Very sparse graph*: with 88% isolated nodes, message passing can help only a small fraction of addresses.
2. *Duplicate edges removed*, so repeated transfers between the same pair are not represented.
3. *Tiny minority test sets* and random (non-temporal) splits.

M. Future Research Directions and Threats to Validity

The authors note that the study uses a single dataset and a single static graph snapshot, and that transfer to other Ethereum datasets, further fraud categories and temporal settings is not established; external validation is needed before deployment.

N. Relevance to Our Proposed Research

(1) Its paired, multiplicity-corrected multi-seed protocol informs Step 9; because we run only three seeds (exact Wilcoxon cannot reach $p < 0.05$ two-sided with three or five pairs), we use a paired block bootstrap instead. (2) The class-weighting ablation supports a cost-sensitive loss. (3) The result that a feature-only MLP is statistically indistinguishable from the best GNN motivates a strong feature-based baseline (B1, Step 6). (4) Its explanation metrics are a reference for future work; explanations are out of scope in this project.

O. Paper-Specific Conclusion and Comparative Positioning

A methodologically careful study that tempers claims of GNN superiority; its static, de-duplicated, very sparse graph contrasts with the dense, temporal multigraphs we study.

4.9 Paper 9: TAML — Translating Latent Representations for Money Laundering Detection

A. Bibliographic Information and Eligibility

- **Full Title:** Translating Latent Representations for Money Laundering Detection
- **Authors:** Ramon Rico, Ioana Hulpus, Stan Leisink, Boyang Zhao, and Yannis Velegrakis
- **Publication Year:** 2026 (IJCAI-26 proceedings)
- **Venue:** *Proceedings of the Thirty-Fifth International Joint Conference on Artificial Intelligence* (IJCAI-26), Special Track on AI4Tech, pp. 6500–6508
- **Publisher:** International Joint Conferences on Artificial Intelligence Organization
- **DOI / Source URL:** 10.24963/ijcai.2026/723; code: https://github.com/rricocuevas1/taml
- **Ranking / Quartile:** CORE2023 **A*** (verified on the ICORE conference portal).
- **Relevance to This Project:** Evaluates on the IBM AML HI-Small data, our discovery set, and explicitly aggregates all transactions between an account pair into one edge.
- **Eligibility Decision: Eligible.** Peer-reviewed 2026 paper at a CORE A* venue; full text examined (open-access proceedings).

B. Research Problem and Motivation

Money-laundering detection involves two related tasks, classifying accounts as suspicious and discovering the laundering sub-network around an account, yet state-of-the-art methods solve one task, or both in separate representation spaces (e.g., Diga, AMAP).

C. Research Objectives and Contributions

1. **TAML**, an encoder-agnostic system that embeds accounts and transactions in one latent space and solves both tasks there.
2. A *translation unit*: inspired by TransE, the embedding of a transaction is trained to act as a translation from its source account's embedding to its destination account's embedding when both accounts are laundering accounts.
3. A *self-translation unit*: an account's own features act as a "self-loop" translation that makes account embeddings label-informative for classification.

D. Related Work and Theoretical Foundation

Related to group-aware AML GNNs (ComGA, GAGNN), sequential approaches that first discover sub-networks and then classify (Diga, AMAP), shallow sub-network discovery (DenseFlow), AML transformers (FraudGT [33], Tab-AML) and the geometric anomaly detector PBAD.

E. Dataset and Data Collection

- **IBM-AML HI-Small** [3]: 5,000,000 transactions over 10 days, built into 10 directed attributed graphs; 515,000 accounts with 26 derived features; 5,893 positive accounts (an account is positive if it takes part in at least one laundering transaction).
- **SAML-D**: 9,504,852 synthetic transactions over 11 months; 856,951 accounts, 2,470 positive.
- **Elliptic++** [43]: 822,942 Bitcoin accounts and 2,868,964 transactions; only the first 42 of 49 time steps are used.

F. Preprocessing and Feature Engineering

A directed edge (v, u) exists if at least one transaction goes from v to u, and its feature vector "contains aggregated information about all transactions from account v to account u within a fixed timespan". Account features are derived from transactions. To prevent leakage, test accounts and their transactions are removed from training and validation; normalisation uses training and validation data only; splits are temporal 70/15/15. Class imbalance is handled by majority-class undersampling in the training set.

G. Model Architecture and Technical Methodology

Any graph encoder (GCN, GIN, GAT, GAT with edge features, FraudGT) produces "t-" and "s-embeddings"; an MLP embeds each transaction as a translation vector. A supervised contrastive loss with learnable "virtual thresholds" enforces the translation relations, and the total loss is a convex combination (λ) of the translation and self-translation losses. At inference, the translation unit explores the K-hop ego-network forwards and backwards to estimate the laundering sub-network, and the account's classification score is the mean self-translation risk over that sub-network.

H. Experimental Setup

Baselines: XGBoost, PBAD, Diga (the authors could not reproduce its reported performance), AMAP, DenseFlow, FraudGT and Tab-AML; results averaged over 5 seeds.

I. Evaluation Metrics

Classification: AUPRC, AUROC and normalised AUPRC (AUNPRC). Sub-network discovery: node-wise precision and recall averaged per class (P0, P1, R1).

J. Results and Quantitative Interpretation

- TAML-FraudGT has the best performance across most datasets, time windows and metrics.
- On IBM-AML its AUPRC on the two test days is **0.39** and **0.65**, against **0.43** and **0.65** for FraudGT, i.e. comparable classification on this dataset.
- For sub-network discovery, TAML-FraudGT reaches P1 ≥ 0.95 on every dataset and time step.

K. Ablation Studies and Component Analysis

On Elliptic++, the full model (AUNPRC 0.64, AUROC 0.92) outperforms variants with only one unit; replacing the translation unit by Diga's shallow sub-network proposer lowers sub-network discovery quality; sharing one backbone between the two units degrades both tasks. TAML also improves every tested encoder over the same encoder trained with majority-class-undersampled binary cross-entropy, and edge-feature-aware encoders (GAT with edge features, FraudGT) perform best.

L. Limitations and Weaknesses

1. *Aggregated edges*: one edge per account pair per timespan removes per-transfer timing, amounts and counts.
2. *Daily snapshots*: the IBM data are processed as 10 daily graphs.
3. *Account-level labels with majority undersampling*: results are not directly comparable with the transaction-level (edge) results on the same data reported by Multi-GNN [1].

M. Future Research Directions and Threats to Validity

The paper does not state explicit future-work directions in the version examined. For our purposes, the main threat is comparability: account-level, undersampled training on daily snapshots differs from the edge-level, full-imbalance streaming setting we study.

N. Relevance to Our Proposed Research

TAML is direct evidence for G1 (explicit per-pair aggregation) and G2 (daily snapshots), and it shows that edge-feature-aware encoders help on IBM-AML. It also confirms that the IBM HI-Small data are an accepted benchmark at a top venue.

O. Paper-Specific Conclusion and Comparative Positioning

An elegant joint classification and sub-network discovery method at a CORE A* venue; its aggregated, snapshot-based representation leaves the per-transfer, event-level setting open.

5 Closest Prior Work and Methodological Positioning

A defensible contribution must be positioned not only against the nine 2025–2026 studies, but also against closely related work that falls outside the 2025–2026 window or whose venue ranking could not be verified (Table 3). The most important conclusion of this positioning is that *a graph neural network that processes directed multigraphs is not, by itself, an open problem in the offline setting*, and that the staleness caused by batching in memory-based temporal GNNs has been studied for generic benchmarks (Section 5.1.7). What remains open is what AML detectors know when they decide, how quality and stoppable value depend on that, and whether the standard benchmark and metric can measure it (Section 5.1.8).

5.1 Detailed Review of Closest Prior Work

5.1.1 Multi-GNN: Provably Powerful GNNs for Directed Multigraphs (Egressy et al., AAAI 2024)

Egressy et al. [1] add three adaptations to standard message-passing GNNs: **multigraph port numbering**, **ego IDs** and **reverse message passing**. They show that together these allow the detection of any directed subgraph pattern. Edge updates (EU) are used as an additional

variant. They treat AML on the IBM synthetic data [3] as *edge classification*, using a 60/20/20 temporal split of transactions with separate train, validation and test graph snapshots.

Reported evidence on IBM AML (arXiv v3 tables and appendix).

- **Minority-class F1 on HI-Small:** XGBoost with graph features (GFs) 63.23%; GIN 28.70%; GIN+EU 47.73%; PNA 56.77%; Multi-GIN+EU 64.79%; Multi-PNA 64.59%; **Multi-PNA+EU 68.16%**. Only the multigraph-adapted variants exceed the XGBoost+GFs baseline; the best margin is 4.93 points.
- **LI-Small** (lower illicit ratio): best result 33.07% (Multi-PNA+EU), versus 27.30% for XGBoost+GFs.
- **Cost on HI-Small (RTX 3090, GIN-based models):** total training time 1,876 s for plain GIN and 12,958 s (≈3.6 h) for GIN with reverse message passing, ports and ego IDs; inference throughput between 18,700 and 58,663 transactions per second.

Relationship and differentiation. Multi-GNN learns on static graph snapshots with neighbourhood sampling and has no continuous-time encoding or node memory. Its reverse message passing motivates our separate incoming and outgoing aggregation, and it is our main multigraph baseline (B2). Its evaluation view is examined in Section 5.1.8.

5.1.2 DIAM: Illicit Account Detection on Cryptocurrency MultiGraphs (Ding et al., CIKM 2024)

DIAM [13] models cryptocurrency transaction networks as directed multigraphs with attributed edges. Its **Edge2Seq** module learns from the sequences of attributes of an account's incoming and outgoing parallel edges. A **Multigraph Discrepancy** module then uses attention-assisted message passing to separate normal from illicit accounts. It is evaluated against 14–15 baselines on four Bitcoin and Ethereum datasets (up to 20 million nodes and 203 million edges).

Relationship and differentiation. DIAM is the closest crypto-specific precedent for per-edge sequence modelling. It is evaluated offline and does not maintain node memory over a stream or report latency.

5.1.3 MEGA-GNN: Message Passing for Multigraphs with Edge Attributes (Bilgi & Atasu)

MEGA-GNN [14] uses a two-stage, neighbour-aware aggregation: messages from parallel edges between the same pair of nodes are first aggregated, and the results are then aggregated across distinct neighbours. The authors argue that collapsing to a simple graph loses information, and that adding auxiliary edge identifiers (as in port numbering) breaks permutation equivariance. MEGA-GNN is permutation equivariant, has the same asymptotic complexity as standard GNNs with edge updates, and reports gains on AML datasets.

Relationship and differentiation. MEGA-GNN is the conceptual reference for the two-stage aggregation in our gated DMT-GNN extension. It uses the same test-graph construction as Multi-GNN (Section 5.1.8).

5.1.4 Graph Feature Preprocessor (Blanuša et al., ICAIF 2024)

The Graph Feature Preprocessor (GFP) [15] is a software library that mines money-laundering subgraph patterns (e.g., fan, cycle and scatter–gather patterns) from a transaction stream in real time. It uses multicore parallelism and a dynamic in-memory graph, and outputs graph

features that substantially improve gradient-boosting models on synthetic AML and Ethereum phishing data.

Relationship and differentiation. GFP shows that *streaming* feature extraction for AML is feasible, but it relies on predefined patterns and a tree-based classifier rather than learned representations. Graph-feature-augmented XGBoost (B1) represents this family in our study.

5.1.5 FraudGT: the Current Accuracy Reference on IBM AML (Lin et al., ICAIF 2024)

FraudGT [33] is a graph transformer for financial fraud detection designed to be simple, effective and efficient; it is also the strongest encoder in TAML [9]. Its paper describes HI-Small as 515,088 accounts and 5,078,345 transactions over 10 days with an illicit rate of 0.102%, split temporally 64/19/17 by transactions.

Reported evidence (minority-class F1, Table 2 of the paper). On HI-Small: FraudGT 69.68%, PE-FraudGT **76.41%** and Multi-FraudGT 76.13%, against 68.60% for Multi-PNA+EU and 63.23% for XGBoost with graph features in the same table. On LI-Small, the best reported value is 47.01% (Multi-FraudGT). The Multi-PNA+EU value differs slightly from the 68.16% reported by Egressy et al. [1], which illustrates why we compare published numbers with stated caveats rather than as exact references.

Relationship and differentiation. FraudGT sets the published accuracy bar on HI-Small, about eight points above Multi-GNN. It is a published reference with the same whole-window view (Section 5.1.8), and we do not compete with it on that view. Whether the bar holds at small horizons, or without the tail, is unknown.

5.1.6 SALT-GNN: Dense Neighbourhoods in AML Graphs (Losavio et al., arXiv 2026)

SALT-GNN [44] (a July 2026 preprint) reports that AML GNNs that look adequate on aggregate F1 degrade on recipient accounts with many incoming transactions, and introduces a recipient-degree-stratified evaluation to expose this. The authors attribute the degradation to multiset non-discriminability, cardinality blindness and attention attenuating weak multi-hop signals, and fuse degree-aware statistical aggregation with attention at every layer. They report 3–6 points higher dense-context F1 on HI-Small and HI-Medium with up to 77% fewer parameters than graph-transformer baselines.

Relationship and differentiation. SALT-GNN overlaps with two elements of our design: the count term in TD-GNN's pooling and our degree-stratified error analysis (Step 11). We therefore claim neither as new. SALT-GNN uses a temporal split and does not study decision timing.

5.1.7 Batch Staleness in Memory-Based Temporal GNNs

Memory-based temporal GNNs such as TGN [37] process events in batches; events in the same batch do not see each other's memory updates. Four recent lines of work study this effect:

- **PRES** (Su et al., ICLR 2024 [16]) characterises this "temporal discontinuity" as the factor that limits batch size in memory-based dynamic GNN *training*, and proposes an iterative prediction–correction scheme with a memory-coherence objective.
- **BADGNN** (Zhou and Ren, arXiv 2025 [17]) also targets training with large batches, using temporal Lipschitz regularisation and adaptive attention adjustment.

- **AnchorRM** (Gharwi et al., arXiv October 2026 [18]; full text read) treats the memory committed at the start of a batch as a fixed anchor and predicts a residual for each event from the causally preceding interactions of the same batch. It reports an average 6.16× training speedup with competitive predictive performance on Wikipedia, Reddit, MOOC and LastFM, on three memory-based backbones.
- **StreamTGN** (Zhang et al., arXiv March 2026 [19]; full text read) is a GPU serving system that keeps node memory resident and updates only affected nodes. It processes batches under a bounded-staleness model in which edges of a batch share a logical timestamp, and it uses a financial institution processing 10,000 transactions per second as a motivating example, but it does not evaluate AML detection.

Relationship and differentiation. Staleness from batching, and mitigations for it, are *not* contributions of this project. Reading AnchorRM in full settled a question Revision 2 left open: its use of causally preceding same-batch interactions covers the learned version of our former intra-block context, so we no longer claim that idea. These works share an assumption we now question for detection: that an event in a batch should be scored as if at its own arrival. A detector that raises its alert when the batch closes may use the whole batch without violating causality, because every event in the batch has already arrived by the decision time. The relevant axis for detection is therefore the decision horizon, of which batching is one implementation.

5.1.8 What Published AML Numbers Measure: Evaluation Views

This subsection reports facts we verified directly in papers and released code. It concerns the protocol, and it does not suggest any error of intent: the authors state their setting clearly.

The standard protocol uses a batch view (verified). Egressy et al. define three snapshots and state that "the test graph contains all the transactions, but only the test indices are used for evaluation", describing this as "the most likely scenario faced by banks and financial authorities who want to classify new batches of transactions" [1]. MEGA-GNN adopts the same construction [14]. The released code matches: in Multi-GNN's `data_loading.py`, the test graph takes the full `edge_index`; its `LinkNeighborLoader` calls pass neither `time_attr` nor a temporal strategy. FraudGT's dataset class builds `e_test` from train, validation and test indices in the same way [33]. A test transfer can therefore draw on transfers that occur after it, up to the end of the test window. In addition, `z_norm` is applied separately to each snapshot, so test features are standardised with statistics that include later transactions.

This is a legitimate batch setting, with an unstated horizon. A bank that scores a closed batch may use everything in it. The issue is that the implied horizon is not stated and differs by transfer: a test edge's look-ahead equals the time remaining until the end of the test window. The result is then cited as evidence of "real-time" suitability, or compared directly with streaming systems.

The most-cited comparison uses unequal views (verified). Blanuša et al. evaluate GFP with batch sizes 128 and 2048 on AML data and reserve an unlimited batch for ETH Phishing, noting that an unlimited batch "essentially corresponds to an offline solution" because "the future transactions are also visible", and that GNN baselines "require the entire dataset to be available" [15]. Their Table 5 reports XGBoost with graph features at 63.23% minority F1 on HI-Small at batch size 128 and 64.77% at 2048. That 63.23% is the value reported by Multi-GNN as the XGBoost+GFs baseline which Multi-PNA+EU (68.16%) beats. So the widely repeated ranking "multigraph GNNs beat graph-feature boosting" compares a near-streaming feature pipeline with a model that sees the whole test window. Whether the ranking survives equal

views is untested, and it is one of our questions (C7, Section 14). Note that GFP's near-equality of 128 and 2048 is a limited test of look-ahead: both batches are minutes wide.

Future information has been shown to help elsewhere. Nguyen et al. [45] make the closest prior argument: in posterior card-fraud detection, verification takes days, so later payments on the card are available before a decision. A bidirectional LSTM using two subsequent transactions outperforms past-only models on over 30 million real transactions (transaction-level AUCPR 0.403 against 0.344 with aggregated features). Their "early card-level" metric scores a model's detection of the first fraud on a card. This establishes that look-ahead is valuable, so we do not claim that. Four things differ here: they study one card's sequence rather than a multi-account graph with coordinated typologies; they vary the number of future transactions rather than a time horizon; they report only the benefit of waiting, not what waiting costs in funds already moved; and they do not address benchmark protocol or adversarial timing.

Adjacent audits and precedents. Daniluk and Dabrowski [46] show a trivial, optimisation-free baseline beating published methods on the Temporal Graph Benchmark, which is a direct precedent for the kind of benchmark finding reported here. Su et al. [47] argue that instance-based metrics such as AP miss temporal structure in errors. Maganti [48] shows on Elliptic, in a seed-matched experiment, that training-time exposure to test-period adjacency produces a 39.5-point paired F1 gap for GraphSAGE, and that the published GNN ranking inverts under a strictly inductive protocol. That work concerns training exposure on node classification with coarse time steps, on a different benchmark. KONTOGRAPH [23] builds a real-time AML pipeline on simulated SEPA Instant data under a 200 ms budget, and uses property-based tests that perturb future events to find leakage in features. That is the closest prior engineering practice, and we adopted the same kind of test; it does not measure performance as a function of horizon, and it does not study AMLworld. Kaufman et al. provide the general formulation of leakage [49]. Based on the searches logged in Appendix B, we found no prior work that measures a quality-versus-horizon curve for graph-based AML, or that reports the benchmark tail effect described next. Absence from our searches is not proof of absence; the search is repeated before any submission.

Documented span versus file contents (verified from the data). Altman et al. report 10 days spanned for the Small datasets and 16 for Medium [3], and FraudGT describes HI-Small as covering 10 days [33]. The released files run longer (18, 17, 28 and 27 days for HI-Small, LI-Small, HI-Medium and LI-Medium). Background traffic stops at exactly the documented boundary and laundering continues afterwards. The Multi-GNN split, which loops over all days in the file, places those days in the test set. Its log line prints only `split[2][:5]`, the first five test days, which may explain why this has not been widely noticed. Measurements are in Section 14.

5.1.9 Adversarial Robustness of Transaction and Temporal Graph Models

- **MemFreezing** (Dai et al., ICML 2025 [50]; full text read): an attacker with knowledge of the model and the graph up to the attack time *injects fake nodes or edges* to push node memories into a stable "frozen" state. It does not alter the timing of existing events.
- **HIA** (Jeon et al., arXiv 2025 [51]; full text read): a restricted black-box attack that ranks node–time pairs with a surrogate and mixes edge injection with targeted deletion.
- **TD-PGD and TDAP** (Sharma et al. [52]): constrain structural perturbations on discrete-time dynamic graphs so they stay close to natural graph evolution.

- **Heijmans et al.** (arXiv, 22 October 2026 [53]; read): the first adversarial study we found on multigraph AML models, from the same group as MEGA-GNN. A white-box attacker rewires receivers of illicit transfers on AMLworld, and adversarial training is the defence. The introduction lists "timing transactions" among documented evasion tactics, but the framework "considers only structural edge rewiring"; perturbing transaction attributes, including timestamps, appears only as future work.

Relationship and differentiation. All of these attackers change the graph. A launderer cannot normally inject accounts into a bank's ledger, nor reroute funds without altering the laundering scheme itself. They do have direct control over *when* they submit. Our threat model assumes only that control: no injected accounts, no changed amounts or counterparties, no gradient access, with funds-flow order preserved. This is a weaker and more realistic adversary. Because Heijmans et al. name timestamp perturbation as a possible extension, we treat this part of the novelty claim as time-sensitive.

Defending by design rather than by training. Adversarial training, the defence in Heijmans et al., needs attack examples and adds training cost. Distributed systems offer another route: Lamport [21] showed that ordering events does not require synchronised physical time. Applied here, evidence defined over logical time (the last k events of an account, the order of events, conservation of funds across them) is unchanged by an adversary who moves events on the wall clock without reordering them. Last-k neighbour sampling is already common in temporal GNNs [37], so the mechanism is not new. Evaluating it as a defence against timing-only evasion, and measuring what it costs on clean data, is what we propose; both remain untested hypotheses (C8, Section 14).

5.1.10 Delay, Money and Alert Capacity

Linking detection to money and time is not new in fraud research, so we do not present stoppable value as a new idea. FRAUDability [54] reports money stolen and time to detection for simulated attacks on individual users. Shen and Kurshan [55] treat alert-threshold selection under processing capacity as a sequential decision problem, using net fraud savings, and adjust thresholds over *operational time*. Our differences are specific: the unit is a multi-transfer laundering attempt in a graph, and the decision variable is *how long after each transfer to look*. A transfer can be re-scored as evidence arrives, which is what makes the budget question new.

Deciding early with partial evidence is well studied for time series (Mori et al. [56] optimise accuracy and earliness jointly), and spending a fixed error budget across repeated looks is standard in sequential clinical trials through alpha-spending functions (Lan and DeMets [57]). We did not find these ideas applied to transaction-graph AML. Repeatedly re-scoring a transfer has the same statistical problem as repeated looks at a trial: every extra look spends false alerts, which is why our policy spends a fixed alert budget across decision points. LAS-GNN [58] models temporal laundering motifs; it addresses how to represent time, not when to decide.

5.1.11 TeMP-TraG and Amatriciana

- **TeMP-TraG** (Gounoue et al., arXiv 2025 [34]): a message-passing mechanism for transaction graphs that prioritises more recent transactions when aggregating messages; it improves four GNNs by 6.19% on average.

- **Amatriciana** (Di Gennaro et al., arXiv 2025, listed by its authors as an ICDM 2024 workshop paper [35]): a temporal GNN that uses the whole transaction graph without splitting it into time-based subgraphs; it reports an F1 of 0.76 and 55% fewer false positives than earlier approaches on a public dataset.

5.2 Comparative Methodological Positioning Matrix

Table 4 compares the closest prior work with this project. Entries are limited to what the cited papers state; "n.r." means not reported in the sources examined.

Table 4: Positioning of This Project Against the Closest Prior Work

Dimension	Multi-GNN [1]	DIAM [13]	MEGA-GNN [14]	GFP [15]	FraudGT [33]	This project
Venue	AAAI 2024	CIKM 2024	TMLR (accepted 2026)	ICAIF 2024	ICAIF 2024	Phase 3
Parallel transfers	Kept; ports, ego IDs, reverse MP	Kept; Edge2Seq over in/out edge sequences	Kept; two-stage aggregation	Mined as subgraph-pattern features	n.r.	Kept as events; direction-separated sampling and pooling
Temporal model	Temporal split, static snapshots	n.r.	n.r.	Streaming, dynamic in-memory graph	n.r.	Explicit view $V(e, H)$ with signed time differences
Learned node memory	No	No	No	No (feature engine)	No	No (gated extension)
Latency / streaming evidence	Offline inference throughput	n.r.	n.r.	Real-time feature extraction	Offline efficiency	Horizon curves, stoppable value, adaptive alerting, service times, queue simulation
Task and data	Edge classification (IBM AML); Ethereum phishing	Account classification (Bitcoin, Ethereum)	Edge-attributed multigraphs incl. AML	Features for GBT (synthetic AML, Ethereum phishing)	Edge classification (IBM AML incl. HI-Small, LI-Small)	Edges on AMLworld and SAML-D; Ethereum conditional

5.3 Synthesis: The Defensible Scientific Niche

This work does *not* claim to be the first GNN for multigraphs (Multi-GNN [1], DIAM [13], MEGA-GNN [14]), the first streaming AML system (GFP [15], KONTOGRAPH [23]), the most accurate model on IBM AML (FraudGT [33]), the first degree-aware AML analysis (SALT-GNN [44]), the first treatment of batch staleness (PRES, BADGNN, AnchorRM, StreamTGN [16–19]), the first demonstration that future information helps fraud detection [45], the first money-denominated fraud metric [54, 55], the first observation that leakage matters [48, 49], or the first adversarial study of multigraph AML [53]. Its claimed contribution, in order of priority, is:

1. an **audit of the AMLworld benchmark** that quantifies how much the post-simulation tail contributes to headline results, with a clean evaluation protocol;
2. a **decision-horizon framework** for graph-based AML: quality against horizon, retrained per horizon, at equal views across feature-based and learned detectors;
3. evidence of a **conflict between F1 and stoppable value** about how long to wait, under an alert budget, and an evaluation of whether re-scoring across horizons beats one good fixed horizon;

4. a **timing-only threat model** with matched-displacement controls, and **timing-invariant detection** as a design-level defence.

Items (2) and (4) are the parts most directly grounded in computer networking: a delay–information trade-off under arrival processes and service limits, and logical rather than physical time as the basis for robust evidence.

6 Thematic Extended Literature Review and Synthesis

This section synthesises the evidence from the nine papers (Section 4) and the closest prior work (Section 5) across five themes.

6.1 Theme 1: Evolution of Temporal Graph Modeling Paradigms

The reviewed work falls into three temporal paradigms.

Static graphs. Ferretti et al. [29], TFGAT [12], Kasula et al. [11], LMAE4Eth [10] and Al-E'mari et al. [8] learn on a single cumulative graph. Time is either ignored or enters only as a feature (LMAE4Eth encodes timestamps inside its transaction text).

Snapshots and stages. TAML [9] builds daily graphs; KAR [7] learns over ten chronological stages; ChronoWave-GNN [27] encodes Elliptic's discrete time steps and operates on snapshots in batch mode; FG-EGCN (Phase 1) uses EvolveGCN over snapshots [26]. Snapshotting introduces artificial boundaries:

1. *Loss of ordering within a snapshot:* transfers inside one window become unordered, so bursts and their sequence are hidden.
2. *Update delay:* a detector that refreshes its view only at snapshot boundaries cannot use the newest transactions until the next boundary. Equally, a detector that decides at a boundary may use everything before it, so snapshotting sets an implicit decision horizon (Step 1).

Continuous-time event streams. GeneralDyG [28] and TcfDyG (Phase 1) [24], like TGAT [36] and TGN [37], treat the data as timestamped events $e_k = (u_k, v_k, t_k, \mathbf{x}_{e_k})$. Time differences are encoded with a learnable functional time encoding [36, 37]:

$$\phi(\Delta t) = \big[\cos(\omega_1 \Delta t + b_1), \ldots, \cos(\omega_d \Delta t + b_d)\big], \tag{2}$$

with learnable frequencies ω_i and phases b_i, so exact inter-event intervals are used without discretisation.

Memory versus recomputation. Memory-based models (TGN) keep a state vector per node and update it as events arrive, which costs memory proportional to the number of accounts but avoids recomputing history. Snapshot models avoid persistent state but must rebuild or re-encode the graph to incorporate new transactions. ChronoWave-GNN's authors explicitly name temporal memory and low-latency streaming as future work [27], and LMAE4Eth's authors name streaming and incremental learning [10].

6.2 Theme 2: Graph Structure vs. Tabular Domain Features

The evidence consistently shows that transaction attributes, not topology alone, carry most of the signal:

- **Structure alone fails.** Ferretti et al. report "very poor results" when the dataset features are removed and only graph structure is used, whereas Chebyshev+Lin on reduced features reaches illicit F1 0.906 [29].
- **Feature-only models are competitive on sparse graphs.** On a graph with 88.03% isolated nodes, GraphSAGE (macro-F1 51.08%) is not significantly better than a feature-only MLP (49.03%; Holm-adjusted $p = 0.074$) [8].
- **Per-transaction semantics matter most.** Removing LMAE4Eth's transaction-semantics language model lowers F1 from 0.8143 to 0.6446, while removing its graph branch lowers it to 0.7632 [10].
- **Strong tabular baselines on IBM AML.** XGBoost with graph features reaches 63.23% minority F1 on HI-Small; standard GNNs are below it (GIN 28.70%, PNA 56.77%), and only multigraph-adapted GNNs exceed it (best 68.16%) [1].

Synthesis. In financial transaction networks, message passing is most useful as a way to *propagate and contextualise transaction attributes* (amounts, timing, counterparties). A credible study must therefore include a strong feature-based baseline (B1) and must keep attributes on every transfer.

6.3 Theme 3: Multigraph Preservation vs. Topological Simplification

Transaction networks are directed multigraphs: the same pair of accounts can transact many times. Yet three of the nine recent studies explicitly simplify them:

1. **KAR** [7] merges parallel edges and sums their amounts.
2. **Al-E'mari et al.** [8] remove duplicate transaction edges.
3. **TAML** [9] defines one edge per account pair whose features aggregate all transactions within a timespan.

LMAE4Eth weights account-graph edges by transaction count and keeps per-transaction detail only in its text sequences [10].

What aggregation can lose. A per-pair sum or count cannot distinguish, for example, ten small transfers made within minutes from one large transfer, nor their timing relative to other activity of the same account (an illustrative case, not an empirical claim). Egressy et al. show that standard message passing needs adaptations to detect arbitrary directed multigraph patterns [1], and Bilgi and Atasu argue that collapsing to a simple graph loses information [14]. Whether keeping per-transfer detail *improves detection inside a memory-based temporal model* is untested; this is our gated extension.

6.4 Theme 4: Extreme Class Imbalance and Optimization Pitfalls

Illicit activity is rare: about 0.07% of transactions in IBM HI-Small [1], about 0.04% of addresses in the XBlock Ethereum network [7], and 0.23% of addresses in BLTE [8]. The reviewed studies handle this in different ways:

- **Cost-sensitive losses:** class weighting raises macro-F1 from 34.41% to 51.12% on BLTE [8]; TFGAT uses a weighted cross-entropy with illegal-class weights of 10 and 6 [12].
- **Semi-supervision:** TFGAT's cyclic pseudo-labelling exploits the roughly 77% unlabelled Elliptic nodes and raises recall from 0.8322 to 0.8390 [12].

- **Re-sampling:** KAR uses 1:1 balanced datasets [7] and TAML undersamples the majority class [9]. Accuracy measured on balanced test sets does not reflect precision at operational prevalence, where even a small false-positive rate produces many alerts.
- **Cautionary example:** Asiri and Somasundaram [32], excluded from the quota, report 0.9856 training accuracy alongside 0.168 validation accuracy, call an AUC of 0.9444 "perfect", and their metric definitions refer to "skin cancer". This illustrates why reviews must check full texts and tables, not abstracts.

Our detectors are evaluated at full prevalence, and alert thresholds are set on full-prevalence validation data. For training, LightGBM uses log-loss and TD-GNN weighted binary cross-entropy on all positives and a fixed-rate sample of negatives (Step 3). Focal loss [59] is retained as an option for the gated DMT-GNN extension.

6.5 Theme 5: Methodological Rigor, Temporal Leakage, and Systems Latency

Four recurring issues limit comparability and deployability:

1. **Random splits over temporal data.** Kasula et al. [11] (random 60–80% splits), LMAE4Eth [10] (random 7:1:2), Al-E'mari et al. [8] (stratified 70/15/15) and the headline result of ChronoWave-GNN [27] (stratified 80/10/10) do not separate training and test data in time. Later transactions can therefore inform earlier predictions.
2. **Temporal splits with non-causal test views.** TAML [9] and Multi-GNN [1] use temporal splits, which is better practice. But a temporal split is not a causal evaluation: in the Multi-GNN protocol, a test transfer can use all later test transfers (Section 5.1.8). Revision 2 of this document did not make this distinction. Maganti shows that a related exposure can invert published rankings on Elliptic [48].
3. **Undocumented splits and borrowed baselines.** Ferretti et al. [29] and TFGAT [12] do not describe their split in the texts examined; TFGAT's baseline numbers are taken from earlier papers.
4. **Limited systems evaluation.** Kasula et al. report 20.8 ms per batch [11], ChronoWave-GNN about 8.4 ms per batch with P95 below 10 ms [27], GFP 30 ms and 148 ms per batch of 128 and 2048 [15], and Multi-GNN reports offline throughput [1]. Among the works reviewed, only KONTOGRAPH sets a latency budget under an operational regime [23]. None reports decision timing against the laundering it is meant to interrupt: how much money has already moved when an alert fires.

Connecting to Computer Networks. Blockchains deliver transactions in protocol-defined batches (blocks; roughly 12-second slots on Ethereum and about 10-minute target intervals on Bitcoin), and instant-payment rails settle within seconds. A monitoring service attached to such a network is a stream processor. Its useful properties are whether it keeps up with the arrival rate, how long a decision takes end to end, and what that delay costs in missed evidence and in funds already gone. These are the quantities our experiments measure.

7 Comprehensive Cross-Paper Comparison Table

Table 5 compares the nine papers on the required dimensions: title, year, dataset, preprocessing, models or techniques, evaluation metrics, results, and limitations or research gap. Results are not directly comparable across rows because datasets, tasks, metrics and split protocols differ; the split protocol is therefore stated in the preprocessing column.

Table 5: Cross-Paper Comparison of the Nine Phase 2 Papers (values as reported in the full texts examined)

#	Title / Venue	Year	Dataset Used	Preprocessing (incl. split)	Models / Techniques	Evaluation Metrics	Reported Results	Limitations / Research Gap
P1	GeneralDyG [28] Proc. AAAI-25	2025	Bitcoin-Alpha, Bitcoin-OTC (trust networks); SWaT, WADI	Temporal k-hop ego-graphs with hierarchical tokens; anomalies injected into test set (1/5/10%, TADDY protocol).	TensGNN (alternating node/edge layers) + temporal-aware Transformer; unsupervised.	ROC-AUC, AP (edge); F1 (node).	B-Alpha AUC 94.01/95.41/**96.28**, AP 24.00/24.02/**26.73**; B-OTC AUC 94.66/94.86/**95.59**, AP 27.89/29.97/27.13; SWaT F1 **85.19**; WADI 60.43.	Injected anomalies, not AML labels; no amounts or explicit multigraph; no latency evaluation.
P2	KAR [7] Electronics	2025	XBlock Ethereum (2.97M addresses, 13.55M transactions, 1,165 phishing)	10 chronological stages; **parallel edges merged (amounts summed)**; 1:1 class balance; random 70/30 within each stage.	GCN, GAT, GATv2, GIN on first-order subgraphs; knowledge-augmented replay with inter-stage edges.	Average accuracy, backward transfer.	With inter-stage edges: KAR and retraining ≥90% average accuracy; without: naïve/replay ≈50%, retraining ≈60%.	Multigraph collapsed; balanced data hides prevalence; stage-level updates; first-order subgraphs.
P3	AML GNN benchmark [29] IEEE Access	2025	Elliptic (203,769 nodes, 234,355 edges, 167 features)	`tx`, `tx+agg`, `tx+agg_red` (highly correlated aggregated features excluded); split not described.	GCN, GAT, GATv2, SAGE, Chebyshev; +final linear layer; +skip connections.	Illicit P, R, F1; averaged F1; training time.	Chebyshev+Lin (`tx+agg_red`): F1 **0.906**, P 0.955, R 0.863, avg. F1 0.983; GATv2 F1 >0.885; structure-only "very poor".	Split undocumented; no parallel transfers or edge amounts; offline only.

#	Title / Venue	Year	Dataset Used	Preprocessing (incl. split)	Models / Techniques	Evaluation Metrics	Reported Results	Limitations / Research Gap
P4	TFGAT [12] Sci. Rep.	2025	Elliptic (over 200K nodes, about 234K edges; 166 features)	Elliptic features used directly; weighted cross-entropy (class weights 1 vs. 10/6); split not described.	2-layer Transformer encoder + GAT; deep cyclic pseudo-label updating (DCPLU).	Illicit P, R, F1; ROC-AUC.	TFGAT F1 0.9006; +DCPLU P **0.9814**, R **0.8390**, F1 **0.9046**, AUC 0.9187; 4.1–5.1 h training.	Split undocumented; baselines copied; global attention and long training unsuited to streaming.
P5	AJKGS-ABCM [11] Discover AI	2025	22,312 labelled Ethereum addresses, 11 classes; 59.47M valid transactions	Etherscan and BigQuery collection; stratified mini-batches; random 60/70/80% splits, 20 repetitions.	GraphSAGE with neighbour attention and attention-guided jumping knowledge.	P, R, micro-F1, macro-F1; time per epoch and per batch.	80:20: P 0.862, R 0.852, micro-F1 0.849, macro-F1 **0.856** (GraphSAGE 0.823); 20.8 ms/batch.	Random splits; static cumulative graph; per-batch timing only.
P6	LMAE4Eth [10] IEEE TIFS	2025	MulDiGraph (XBlock), B4E (597,258 accounts), SPN (496,740 accounts)	Transactions serialised as text (amount, direction, timestamp); count-weighted edges; random 7:1:2 split.	TxCLM language model + masked account-graph autoencoder (MAGAE) + cross-attention fusion.	P, R, F1, balanced accuracy.	F1: MulDiGraph **0.8960**, B4E **0.8143**, SPN **0.8002**; w/o LM 0.6446, w/o graph 0.7632 (B4E).	Batch mode, no incremental updates (authors); random split; per-transfer detail only in text.
P7	ChronoWave-GNN [27] Sci. Rep.	2026	Elliptic (stated as 45,576 labelled nodes); cross-checks on AMLSim and Ethereum phishing	Level-2 Haar wavelet features + sinusoidal time-step encoding; stratified 80/10/10 (plus rolling-window test).	TGAT+ (3-layer TransformerConv), label smoothing, AdamW.	Accuracy, P, R, F1; latency per batch, P95, throughput.	Acc. **0.9802**, F1 **0.9799**; ≈8.4 ms/batch, P95 <10 ms; >5.5M nodes/s.	Snapshot batch mode (authors); random headline split; dataset statistics inconsistent with canonical Elliptic.

#	Title / Venue	Year	Dataset Used	Preprocessing (incl. split)	Models / Techniques	Evaluation Metrics	Reported Results	Limitations / Research Gap
P8	Explainable GNNs [8] Future Internet	2026	BLTE multi-class (73,033 addresses, 23 features, 6,588 edges)	**Duplicate transaction edges removed**; inverse-frequency class weights; stratified 70/15/15, 10 seeds.	GraphSAGE, GCN, GAT, Chebyshev GCN; MLP, RF, XGBoost; GNNExplainer; Wilcoxon + Holm.	Accuracy, macro and weighted F1; fidelity$^-$, sparsity, stability.	GraphSAGE macro-F1 **51.08±1.54%** vs. MLP 49.03±2.16% (adj. $p = 0.074$); unweighted 34.41%; joint fidelity$^-$ 31.88%.	88% isolated nodes; de-duplicated static graph; tiny minority test sets; random splits.
P9	TAML [9] Proc. IJCAI-26	2026	IBM-AML HI-Small, SAML-D, Elliptic++	**One aggregated edge per account pair per timespan**; 10 daily graphs (IBM); derived account labels; majority undersampling; temporal 70/15/15.	Translation + self-translation units over any encoder (best: FraudGT); translation-guided sub-network search.	AUPRC, AUROC, AUNPRC; sub-network P0, P1, R1.	IBM-AML AUPRC **0.39/0.65** (two test days) vs. FraudGT 0.43/0.65; sub-network P1 $\geq$0.95 on all datasets.	Aggregated edges; daily snapshots; account-level undersampled setting not comparable with edge-level results.

7.1 Cross-Study Capability Matrix

Table 6 summarises which capabilities each study demonstrates. ✓ = yes; ∼ = partially; × = no; ? = not reported in the text examined. The last row lists this project's capabilities; those not yet demonstrated are marked ∼.

Table 6: Capability Matrix of the Reviewed Studies, the Closest Prior Work, and This Project

Paper / System	C1: Multi	C2: Time	C3: Mem	C4: Stream	C5: Imbal	C6: Real	C7: Causal	C8: XAI
P1: GeneralDyG [28]	×	✓	×	×	n/a	✓	?	×
P2: KAR [7]	×	∼	×	×	×	✓	∼	×
P3: Ferretti et al. [29]	×	×	×	×	?	✓	?	×
P4: TFGAT [12]	×	×	×	×	✓	✓	?	×
P5: Kasula et al. [11]	×	×	×	∼	∼	✓	×	×
P6: LMAE4Eth [10]	∼	∼	×	×	✓	✓	×	×
P7: ChronoWave-GNN [27]	×	∼	×	∼	✓	✓	∼	∼
P8: Al-E'mari et al. [8]	×	×	×	×	✓	✓	×	✓
P9: TAML [9]	×	∼	×	×	×	✓	✓	✓
Prior: Multi-GNN [1]	✓	∼	×	∼	✓	✓	✓	×
Prior: DIAM [13]	✓	?	×	?	?	✓	?	×
This project	✓	✓	×	✓	∼	∼	✓	×

Capabilities: C1: parallel transfers kept as separate edges in learning; **C2:** use of transaction time (✓ continuous time; ∼ discrete steps, stages or timestamps as features); **C3:** learned node memory updated over time; **C4:** latency or streaming evaluation (∼ per-batch latency or offline throughput only); **C5:** imbalance handled without 1:1 down/undersampling (∼ stratified batch sampling; n/a for unsupervised); **C6:** evaluated on real public blockchain data; **C7:** temporal train/test separation (∼ partly, e.g. chronological stages with random splits inside, or a supplementary rolling-window test; this does not imply causal test-time views, Section 5.1.8); **C8:** instance-level explanations (∼ attention visualisations or qualitative only).

8 Research Gap Identification and Synthesis

Table 7 synthesises the research gaps using evidence from three primary papers chosen because each works on a cryptocurrency or AML transaction network at account or transaction level and documents its graph construction, temporal processing and split protocol in enough detail to support every gap:

1. **Paper 1 — KAR (Tian & Zhang, *Electronics* 2025) [7]:** continual learning on the XBlock Ethereum network (our conditional extension).
2. **Paper 2 — LMAE4Eth (Jia et al., *IEEE TIFS* 2025) [10]:** Ethereum fraud detection on three real datasets, including XBlock.
3. **Paper 3 — TAML (Rico et al., IJCAI 2026) [9]:** AML classification and sub-network discovery on IBM HI-Small (our discovery set), SAML-D and Elliptic++.

Corroborating evidence from the other six papers and the closest prior work is given in Section 8.1.

8.1 Detailed Justification of the Four Research Gaps

1. **G1 (Structural multigraph preservation):**
   - *What has been done:* offline GNNs that keep parallel edges exist (Multi-GNN [1], DIAM [13], MEGA-GNN [14]).
   - *Existing approaches in 2025–2026 AML work:* parallel transfers are summed [7], de-duplicated [8], aggregated per pair [9] or reduced to counts [10].

Table 7: Research-Gap Synthesis Table (evidence from three primary 2025–2026 papers)

Research Gap	**Evidence from Paper 1: KAR [7]**	**Evidence from Paper 2: LMAE4Eth [10]**	**Evidence from Paper 3: TAML [9]**	**Proposed Research Opportunity**
G1 (Structural): parallel transfers collapsed before learning	Each snapshot "is a multigraph"; parallel edges are merged and their amounts summed into a simple weighted graph.	Account-graph edges are weighted by transaction count; individual transfers enter only as text tokens, not as edges in message passing.	One directed edge per account pair, with features aggregating all transactions within a fixed timespan.	Keep every transfer as a directed event in features and sampling. Two-stage aggregation inside a memory-based model is tested only if gate G2 finds a gap worth closing.
G2 (Decision timing): no horizon or stoppable-value characterisation	Learning proceeds stage by stage over 10 chronological partitions; updates happen at stage granularity.	Batch mode on a graph snapshot; the authors list streaming and incremental learning as future work.	IBM data processed as 10 daily graphs; scalability is reported as training/inference runtime versus volume, not arrival-driven latency.	An explicit decision horizon H; detection quality, stoppable value and end-to-end delay as functions of H and offered load; alert budgets spent across horizons.
G3 (Evaluation validity): protocols that hinder comparison	1:1 balanced datasets with random 70/30 splits inside each stage; accuracy and backward transfer only.	Random 7:1:2 node split in the main experiments; no temporal split.	Temporal split, but account-level derived labels with majority undersampling, so IBM results are not comparable with edge-level results on the same data.	One chronological protocol with a stated view, reported separately for the test core and tail, full-prevalence evaluation, a strong feature-based baseline, paired bootstrap, and frozen hypotheses confirmed on unseen datasets.
G4 (Forensic, deferred): explanations for streaming detectors	No instance-level explanations reported.	No instance-level explanation analysis reported in the version examined.	The discovered sub-network serves as an explanation, but it is computed on aggregated daily snapshots.	Out of scope: formal temporal explainers remain deferred.

- *Limitations:* aggregation removes per-transfer amounts, timing and counts within a pair.
- *What remains uninvestigated:* whether per-counterparty, direction-specific aggregation of individual transfers adds value *inside a continuous-time memory model*, where memory already summarises history.
- *Importance:* the IBM simulator generates typologies built from many related transfers (fan-in/out, scatter–gather, cycles) [3], and LMAE4Eth's ablation shows per-transaction information is highly informative [10]; whether this transfers to graph aggregation is an empirical question.

2. **G2 (Decision timing and evaluation views):**

- *What has been done:* per-batch latency on one GPU (ChronoWave-GNN [27], Kasula et al. [11]), offline throughput (Multi-GNN [1], FraudGT [33]), real-time feature extraction with batch sizes up to 2048 (GFP [15]) and a point-in-time pipeline under a latency budget (KONTOGRAPH [23]). For generic temporal graphs, batch staleness has been characterised and mitigated [16–19].
- *Existing approaches:* the leading edge-level AML models are evaluated on a test graph containing the whole test window [1, 14, 33], or on snapshots and stages [7, 9, 10, 27].
- *Limitations:* the horizon each result assumes is rarely stated and is non-uniform across transfers; published comparisons mix views (Section 5.1.8); detection delay relative to the laundering it should interrupt is not reported.
- *What remains uninvestigated:* detection quality as a function of an explicit horizon with retraining per horizon, for feature-based and learned detectors; how much value can still be stopped at the first alert; and whether alerts should be spent at one horizon or across several.
- *Importance:* under instant-payment rules [22], funds move within seconds, so a detector's usefulness depends on what it knows when it decides and what remains to be stopped by then.

3. **G3 (Evaluation validity and generalisation):**

- *What has been done:* evaluations on single synthetic or single real datasets.
- *Existing approaches:* random or stratified node splits [8, 10, 11, 27], undocumented splits and copied baselines [12, 29], balanced re-sampling [7, 9].
- *Limitations:* random splits let later information inform earlier predictions; balanced sampling hides the false-positive volume at real prevalence; single datasets leave external validity untested.
- *What remains uninvestigated:* the same detectors on several AMLworld datasets, an independent simulator (SAML-D [60]) and a real blockchain under one chronological, full-prevalence protocol with an explicit view.
- *Importance:* without comparable protocols, reported gains cannot be attributed to the method rather than to the evaluation.

4. **G4 (Forensic explainability for streaming models; deferred):**

- *What has been done:* rigorous post-hoc explanation on static graphs (fidelity, sparsity and stability for GNNExplainer [8, 41]); sub-networks as explanations [9].
- *Limitations:* post-hoc explainers optimise a mask per instance and were designed for static graphs; with node memory, part of the evidence lies in the memory state, which such masks do not capture.

- *What remains uninvestigated:* faithful explanation of memory-based temporal detectors.
- *Scope decision:* this is a research problem in its own right and remains deferred; no quantitative work on explanations is planned.

5. **G5 (Benchmark integrity; added in Revision 3):**

- *What has been done:* AMLworld [3] is the standard edge-level AML benchmark, used by Multi-GNN, MEGA-GNN, FraudGT, GFP, TAML and SALT-GNN [1, 9, 14, 15, 33, 44].
- *Limitations (verified):* the released files extend past the documented span; the extra days hold almost no background traffic, and they fall in the standard test split.
- *What remains uninvestigated:* how much of each published result comes from that period, and how results rank without it.
- *Importance:* model rankings guide the field. A benchmark in which a trivial date rule scores in the range of published models cannot separate them reliably on the affected share of positives.

6. **G6 (Timing robustness; added in Revision 3):**

- *What has been done:* structural attacks on temporal and multigraph models [50–53].
- *Limitations:* every attack we found modifies the graph. Timing appears only as motivation or future work [53].
- *What remains uninvestigated:* whether an adversary who changes only submission times can suppress detection, and whether detector design (horizon, tie handling, logical-clock evidence) changes this without adversarial training.
- *Importance:* timing is the one control a launderer always has. Attacks built on it cost almost nothing to mount.

9 Refined Research Gap and Problem Formalization

Combining the Phase 1 problem with the Phase 2 evidence, the code and data audits, and the closest prior work, we answer the six framing questions and state the refined research gap.

9.1 Systematic Six-Question Justification

1. **What has already been done?** GNNs have been applied to Bitcoin and Ethereum AML and fraud detection with static graphs [8, 11, 12, 29], transaction-semantics models [10], continual learning over stages [7], snapshot-based temporal transformers [27] and joint classification–sub-network discovery [9]. Offline multigraph GNNs [1, 13, 14] and a streaming feature engine [15] have been developed independently. On IBM HI-Small, multigraph-adapted GNNs report up to 68.16% minority F1 against 63.23% for XGBoost with graph features [1], and graph transformers up to 76.41% [33]. Outside AML, batch staleness has been studied for training and serving [16–19]. Structural adversarial attacks exist for temporal and multigraph models [50, 51, 53].

2. **What approaches currently exist?**

- *Static and snapshot GNNs or transformers* on single, cumulative or daily graphs [9, 11, 12, 27, 29].
- *Offline multigraph GNNs* that keep parallel edges and evaluate with a whole-test-window view [1, 14, 33].
- *Streaming feature engines* feeding tree models [15] and a point-in-time real-time pipeline [23].

- *Continuous-time temporal GNNs* with node memory (TGN [37]) or temporal ego-graphs (GeneralDyG [28]).

3. **What are their limitations?**

- *Unstated evaluation horizons:* published AML results mix near-streaming and whole-window views and are compared directly (Section 5.1.8).
- *A benchmark with a tail:* the standard test split includes a period after background traffic ends (Section 5.1.8).
- *Metrics that ignore time:* edge F1 credits an alert equally whether it fires before the money moves or days afterwards.
- *Aggregated multigraphs* in recent pipelines [7–10] and *protocols that hinder comparison* [7–12, 29].

4. **What remains insufficiently investigated?** Among the works reviewed, none

   (a) measures AML detection quality as a function of an explicit decision horizon, retraining at each horizon;

   (b) quantifies how much published AMLworld performance comes from the post-simulation tail;

   (c) measures how much laundered value can still be stopped when the first alert fires, under an alert budget, or allocates that budget across decision points;

   (d) tests a timing-only adversary with controls that separate the attack's structure from mere displacement; or

   (e) compares feature-based and learned detectors at equal horizons.

5. **Why is this gap important?** Instant-payment regulation removes the batch window [22], and laundered funds keep moving while a detector waits. If published accuracy depends on seeing the future, and if the headline metric rewards decisions too late to act upon, then the field may be ranking detectors by a property deployments cannot use.

6. **How will our research address it?**

   (a) A **horizon-parameterised view** $V(e, H)$ used identically by a feature engine and a GNN sampler, with property tests that perturb data beyond the view.

   (b) A **benchmark audit** on all four AMLworld Small and Medium datasets and, for independence from one simulator, on SAML-D [60].

   (c) The **stoppable-value metric** and an **adaptive alert policy** tuned on validation only.

   (d) A **timing-only attack study** with matched-displacement controls, and a **timing-invariant detector**.

   (e) **Confirmatory testing** on data not modelled when the hypotheses were frozen.

9.2 Formal Project Statements

- **Concise Final Research-Gap Statement:** *Published AML detectors are evaluated with an unstated, non-uniform view of future transactions, on a benchmark whose test split extends past the end of its background traffic, using metrics that ignore when an alert fires relative to the money it should stop. Consequently, it is unknown how detection quality depends on decision horizon, how much reported performance deployments can actually use, how to allocate limited alerts over time, or whether an adversary who controls only submission times can evade detection.*

- **Refined Research Problem Statement:** *For a transfer scored with evidence up to a horizon H after it occurs, how do detection quality and the value of laundered funds that can still be stopped vary with H; how should a fixed alert budget be spent across horizons; and how robust are these detectors to an adversary who changes only when transfers are submitted?*
- **Research Aim:** To establish decision horizon as an explicit parameter of AML evaluation, to measure what it costs and yields on transaction benchmarks, and to derive alerting and detector-design guidance that holds under timing-only attack.
- **Specific Research Objectives:**
  1. Implement the horizon view for feature-based and learned detectors, with tests that prove nothing beyond the view is used.
  2. Audit AMLworld's temporal structure and quantify its effect on published-style evaluations, including the official Multi-GNN.
  3. Measure quality and stoppable value against horizon, retraining at each horizon, by typology.
  4. Design and evaluate an adaptive multi-horizon alert policy against the best fixed horizon, under alert budgets.
  5. Evaluate timing-only attacks with matched controls, and compare view-based and logical-clock defences.
  6. Measure service time and queueing behaviour, and report end-to-end decision delay under load.
- **Research Questions (RQs, in order of priority):**
  - **RQ1 (Decision horizon; primary):** How do minority F1, PR-AUC and stoppable value change with decision horizon H, for feature-based and learned detectors, overall and by typology? Do F1 and stoppable value agree on the best H?
  - **RQ2 (Benchmark integrity):** How much do AMLworld's post-simulation days contribute to standard results, and how do model rankings change without them and under equal views?
  - **RQ3 (Adaptive alerting):** Does spending an alert budget across horizons stop more value than the best fixed horizon, and at what computational cost?
  - **RQ4 (Timing robustness):** Can a timing-only adversary suppress detection beyond a matched-displacement control, and does evidence defined over logical time resist this at an acceptable cost on clean data?

  Gated extension (not a research question unless the gate in Section 13 passes): whether node memory and two-stage per-counterparty aggregation (DMT-GNN) close a measured gap at small horizons. *Conditional extension:* the real Ethereum network (D2).
- **Proposed Contributions:** (1) the decision-horizon framework and open-source implementation; (2) a reproducible audit of AMLworld and a clean evaluation protocol; (3) the stoppable-value metric and adaptive alert policy, with controlled evidence, positive or negative; (4) a timing-only threat model and evidence on timing-invariant detection.
- **Scope and Boundaries:** Edge-level binary classification on AMLworld Small and Medium, and SAML-D for simulator independence, with real Ethereum data as a conditional extension. Stoppable value is an accounting measure on simulated attempts: it assumes an alert can stop later transfers in the same attempt, and it can count the same funds across hops. Out of scope: mempool detection, privacy coins, cross-chain tracing, operational evasion guidance, and any real institution's systems.

- **Assumptions and Constraints:** Simulator labels are treated as ground truth; synthetic results do not establish real-world effect sizes. Timestamps have one-minute resolution, so real-time performance is reported as an interval between the strict-past and same-minute views. Experiments run across free cloud notebooks, the university GPU and team hardware, not on any single machine (Section 13).

10 Proposed Research Methodology: The 12-Step Framework

The methodology turns the refined gap into a reproducible experimental pipeline. Each step states its Objective, Input, Procedure, Output, Technical Justification, Literature Support, and Risks & Controls. Figure 1 (Section 12) shows how the steps connect.

10.1 Step 1: Define the Research Input

- **Objective:** Specify the data representation, the task and the decision semantics precisely.
- **Input:** Transaction records with timestamp, sender and receiver account, amounts, currencies, payment format and a per-transaction label; a separate file lists laundering attempts by typology [3].
- **Procedure:**
  1. Represent each dataset as a directed multigraph of events $e = (u, v, t, \mathbf{x}_e)$. Parallel transfers are never merged.
  2. Define the task as edge classification: predict $y_e \in \{0, 1\}$ for each transfer.
  3. Define the **decision horizon** $H \geq 0$ and the detector's view of event e:

$$V(e, H) = \begin{cases} \{e' \neq e : t_{e'} < t_e\} & H = 0 \quad \text{(strict past)} \\ \{e' \neq e : t_{e'} \leq \min(t_e + H,\ \text{end}(e))\} & H > 0, \end{cases} \tag{3}$$

     where $\text{end}(e)$ is the end of e's split, so training and validation views never see later splits. The decision for e is taken at $d_e = t_e + H$.
  4. **Tie semantics.** Timestamps have one-minute resolution, so the order of same-minute events is unknown. $H = 0$ treats them as invisible (a lower bound on real-time capability); $H = 1\,\text{s}$ makes them all visible (an upper bound). Real-time performance is reported as this interval.
  5. The standard Multi-GNN protocol corresponds to $H = \infty$, with an implicit per-transfer horizon $\text{end}(e) - t_e$.
- **Output:** Schemas and a single view definition used by every model.
- **Technical Justification:** One definition lets a feature pipeline and a GNN be compared at equal information, which published comparisons do not do (Section 5.1.8).
- **Literature Support:** Egressy et al. [1], Blanuša et al. [15], Kaufman et al. [49].
- **Risks & Controls:** *Risk:* an implementation silently sees beyond the view. *Control:* property tests rewrite all events outside $V(e, H)$ and require byte-identical features and samples (Step 3).

10.2 Step 2: Data Collection

- **Objective:** Obtain and verify the benchmarks.
- **Input:** Public dataset releases.
- **Procedure:**
  1. **AMLworld** [3]: HI-Small (discovery set); LI-Small, HI-Medium and LI-Medium (confirmatory). HI-Small matches the published counts: 5,078,345 transfers and 515,088 accounts. Its illicit rate is 0.102%, matching FraudGT's figure rather than the 0.07% in Multi-GNN's table.
  2. **SAML-D** [60] (confirmatory, not yet obtained): a synthetic transaction-monitoring dataset from an independent simulator, used so that findings do not rest on one generator's timing model.
  3. **XBlock Ethereum** [61] (conditional): real transfers with account-level labels; used only for horizon curves on a pre-chosen window, since labels are per account.
  4. Every raw file is verified by row count and SHA-256. This is necessary: one download endpoint truncated a file silently while reporting success.
- **Output:** Verified raw tables, checksums and a generated data card.
- **Technical Justification:** AMLworld provides typologies and the published reference results; a second simulator and real data address external validity, the largest threat to any synthetic-data finding.
- **Literature Support:** Altman et al. [3], Oztas et al. [60], van den Beukel et al. [62], Wu et al. [61].
- **Risks & Controls:** *Risk:* findings reflect one simulator's scheduling. *Control:* the independent simulator, and the rule that synthetic effect sizes are not claimed as real-world effect sizes.

10.3 Step 3: Data Preprocessing

- **Objective:** Partition chronologically under two explicit protocols, with automated leakage checks.
- **Input:** Raw tables from Step 2.
- **Procedure:**
  1. *Split:* reproduce Multi-GNN's whole-day rule (brute-force search over day cuts for proportions closest to 60/20/20). On HI-Small this yields days 0–5, 6–7 and 8–17, or 64.0/19.0/17.0% of transfers, matching FraudGT's reported 64/19/17 and explaining the apparent disagreement with Multi-GNN's stated 60/20/20.
  2. *Two protocols:* **Released** uses all days in the file, as published work does. **Documented** removes events after the documented span before evaluation. Results are always reported for the test set's *core* and, where it exists, its *tail*.
  3. *Rolling-origin folds* (planned): several train/validation/test cuts inside the documented span, because a single split leaves only 86 attempts wholly in the HI-Small test window.
  4. *Scaling and conversion:* USD exchange rates and vocabularies are estimated on the training split only. Tree models need no scaling; GNN amount scaling uses training statistics, unlike the per-snapshot normalisation in the reference code.

5. *Imbalance:* no resampling at evaluation. Training uses all positives and a random sample of negatives (rate fixed in advance). *This changes Revision 2*, which specified no negative sampling: at 5M events it would make per-horizon retraining impractical, and thresholds are set on full-prevalence validation data, so operating points are not distorted.
6. *Automated leakage gate:* tests rewrite (i) all events at or after t_e for $H = 0$, (ii) all events after $t_e + H$ for finite H, and (iii) all events after the split end, and require unchanged features and neighbour samples. A brute-force reference implementation written from the definition must match the fast implementation on random streams with ties and self-loops.

- **Output:** A canonical event table, attempt tables and a passing test report.
- **Technical Justification:** Reporting both protocols keeps comparability with published work while exposing the tail effect. The future-perturbation test catches any leak mechanism, rather than only those a reviewer thought of [23].
- **Literature Support:** Egressy et al. [1], Lin et al. [33], Abolfadl [23].
- **Risks & Controls:** *Risk:* an honest test can pass on a vacuous stream. *Control:* mutation checks confirm the reference disagrees with the engine under wrong semantics.

10.4 Step 4: Feature Engineering and Representation

- **Objective:** Build features computed only from $V(e, H)$.
- **Input:** Canonical events.
- **Procedure:**

1. *Event attributes:* log amounts, USD value, currencies, payment format, same-currency and self-loop flags, hour of day.
2. *Past block:* for each endpoint and direction, and each window $W \in \{6, 24, 72\}$ h: count, log-sum USD, distinct counterparties, time gap, and standard deviations of log-amount and event time.
3. *Forward block:* the same statistics over the forward part of the view; all zero at $H = 0$.
4. *Pair block:* past and forward count and value for (u, v) and (v, u).
5. *Motif block:* two-paths $u \to w \to v$ in temporal order; three-cycles closed by e and opened by e; convergent siblings (recipients of u that share a downstream account with v, and the mirror case). Scans over high-degree accounts use documented caps.
6. *GNN inputs:* per-edge log amount, categorical embeddings, and signed log time difference relative to the seed.
7. *Logical-clock block (proposed, not yet implemented):* the same statistics over the account's last k events rather than the last W hours; order-based features (rank of a transfer among the account's recent events, whether an inflow precedes an outflow); and conservation features (running net USD through the account over those events). None of these depend on wall-clock gaps.

- **Output:** Feature matrices per horizon.
- **Technical Justification:** The feature families follow those that drive GFP's results (fan, cycle, scatter–gather and vertex statistics [15]). A strong causal baseline matters: a weak one would overstate the value of look-ahead. As a check, our same-minute view reaches a test F1 at or above GFP's published batch-128 value (Section 14).
- **Literature Support:** Blanuša et al. [15], Egressy et al. [1], Losavio et al. [44].

- **Risks & Controls:** *Risk:* simulator-specific features (e.g. hour of day, payment format) drive results. *Control:* ablations remove them (Step 10).

10.5 Step 5: Proposed Methods

- **Objective:** Define the horizon-aware detector, the alert policy and the learned model.
- **Input:** Views and features from Steps 1 and 4.
- **Procedure:**

  1. **Horizon-specific detectors.** A detector is trained separately at each H using that H's view, so that a change in quality cannot be a train/serve mismatch.
  2. **Adaptive multi-horizon alerting.** A transfer is re-scored at checkpoints $H_1 < \cdots < H_m$, giving scores $s_j(e)$. It alerts at the first checkpoint it clears:

  $$J(e) = \min\{\, j : s_j(e) \geq \tau_j \,\}, \qquad \text{alert time } = t_e + H_{J(e)}. \tag{4}$$

  Given a daily budget B, fractions α_j with $\sum_j \alpha_j = 1$ are spent in checkpoint order, and τ_j is set on validation so that transfers alerting first at j consume α_j of the budget. The allocation α maximises validation stoppable value by exhaustive search on a simplex grid; test data influences neither α nor τ. Fixed-horizon detectors are the corners of this simplex. The idea follows alpha-spending [57], with false alerts in place of type-I error.

  3. **Temporal directed GNN (TD-GNN).** A two-hop model on a deterministic sampler that returns the nearest events in time within the seed's view. First-hop and second-hop lists are separated by direction, as in reverse message passing [1]; pooling combines mean, max and log-count to keep cardinality information [44]. Second-hop lists obey the *seed's* view, so a neighbour cannot contribute evidence unavailable at the decision time. The architecture is identical at every horizon; slots a horizon cannot fill are padded.
  4. **Timing-invariant detection (proposed).** A detector restricted to the logical-clock block of Step 4 and the event attributes, compared with the wall-clock detector on clean and attacked data. It needs no attack examples and no adversarial training. Its clean-data cost is the price of robustness, and it may be large, because wall-clock features carry real signal.
  5. **Gated: memory and two-stage aggregation.** Revision 2's DMT-GNN (GRU memory, per-counterparty Stage 1, cross-counterparty Stage 2) is built only if TD-GNN shows a modelling gap at small horizons relative to the feature baseline.

- **Analytical results (proved directly from the definitions):**

  1. *Random boundaries do not stop bursts.* If a detector closes units of length Δ with uniformly random phase, a burst of span s crosses a boundary with probability $\min(1, s/\Delta)$. A one-minute burst against a one-hour unit is split with probability $1/60$. Boundary jitter, proposed in Revision 2, is therefore not a defence against tight bursts.
  2. *Evidence windows set an evasion rate limit.* If a detector's evidence about an account covers a past window W and horizon H, transfers of that account can share no view only when consecutive gaps exceed $W + H$. Evading all shared views with n transfers takes more than $(n-1)(W+H)$. The detector thus imposes a rate limit of one transfer per $W + H$ on fully evasive activity. This bound applies to window-bounded evidence; memory models have no hard window.
  3. *Logical-clock evidence resists stretching.* Under a non-decreasing re-timing, the relative order of an attempt's transfers is preserved, so a sibling leaves an account's last-k window

only if at least k of the account's events fall between them. For a dedicated mule account with little other traffic, no amount of stretching removes the evidence. The bound does not protect high-traffic accounts, whose background events can displace siblings, and order against background events can still change. How much this matters in practice is an empirical question (C8).

- **Output:** Tested implementations of the view, feature engine, sampler, TD-GNN and policy.
- **Technical Justification:** Per-horizon retraining gives a fair curve. The policy tests whether flexibility in *when* to spend alerts is worth more than picking one horizon. TD-GNN gives a learned model whose view can be restricted exactly, which the reference code does not support directly.
- **Literature Support:** Lan and DeMets [57], Mori et al. [56], Lamport [21], Egressy et al. [1], Rossi et al. [37].
- **Risks & Controls:** *Risk:* TD-GNN underperforms published GNNs, which would undermine claims about GNNs generally. *Control:* the official Multi-GNN is audited under cadence replay (Step 7b), and TD-GNN is never presented as Multi-GNN.

10.6 Step 6: Baseline Methods and Controls

- **Objective:** Test each claim against strong, relevant comparators.
- **Input:** The same splits and views.
- **Procedure:**
  1. **B0: date rule (diagnostic only).** Flag every transfer in the tail. It has no learning and uses knowledge of the tail, so it is not a detector; it measures the weight of the tail in the metric.
  2. **B1: LightGBM on our features at each H.** The strong feature-based reference.
  3. **B2: official Multi-GNN**, unmodified training code, audited by region and by cadence replay. If it cannot be run, published values are labelled as not reproduced; a re-implementation is never presented as Multi-GNN.
  4. **B3: TD-GNN** at each H.
  5. **B4: GFP with XGBoost** [15] (Linux only) on the Multi-GNN split, as the published streaming reference.
  6. **Published references:** FraudGT and Multi-GNN values, labelled as published and not re-run, with their view stated.
  7. **Policy controls:** best fixed horizon selected on validation, and each fixed horizon separately.
  8. **Attack controls:** matched-displacement rigid shifts (Step 7d).
- **Output:** One codebase evaluating all detectors under one protocol.
- **Technical Justification:** B1 and B3 compare families at equal views; B2 connects findings to the published model; B0 quantifies the benchmark effect independently of any model.
- **Literature Support:** Egressy et al. [1], Blanuša et al. [15], Lin et al. [33], Nguyen et al. [45].
- **Risks & Controls:** *Risk:* the official code does not run on current library versions. *Control:* a pinned environment and a time box; the harness is written against the official functions.

10.7 Step 7: Experimental Design

- **Objective:** Answer RQ1–RQ4 and measure systems behaviour.
- **Input:** Detectors and protocols.
- **Procedure:**
  1. **Step 7a – Horizon sweep (RQ1):** $H \in \{0, 1\,\text{s}, 1\,\text{min}, 10\,\text{min}, 1\,\text{h}, 6\,\text{h}, 24\,\text{h}, 72\,\text{h}, \infty\}$; 3 seeds; retrained at every H; thresholds set on validation; both protocols.
  2. **Step 7b – Benchmark audit (RQ2):** volume by day on all datasets; B0 on each; region metrics for all detectors; official Multi-GNN weights replayed at cadences of 1, 6 and 24 h, where the graph holds only edges up to the end of the current bucket. Replay shows the effect of the view at fixed weights; the fair per-horizon comparison uses retrained models.
  3. **Step 7c – Adaptive alerting (RQ3):** checkpoints $\{1\,\text{s}, 10\,\text{min}, 1\,\text{h}, 6\,\text{h}, 24\,\text{h}\}$; budgets $B \in \{100, 500, 2000\}$ alerts/day; α on a 0.1 grid, chosen on validation; evaluated once on test.
  4. **Step 7d – Timing attack (RQ4):** detectors are trained on clean data with thresholds fixed on clean validation. Only attempts starting in the test window are re-timed, with budget $W \in \{1, 6, 24\}$ h. **Burst:** $t' = \text{clip}(c, t - W, t + W)$ with c the attempt median. **Stretch:** $t' = t + W(2r - 1)$ with r the rank within the attempt. **Controls:** a rigid shift with the same mean displacement and random sign, which preserves internal gaps. Maps are non-decreasing, so funds-flow order holds; times stay on the minute grid; only times change. Features are recomputed from the perturbed stream. The same attacks are applied to wall-clock, logical-clock and TD-GNN detectors.
  5. **Step 7e – Service and queueing:** measure feature-plus-scoring time for batch sizes $n \in \{1, \ldots, 10^4\}$ on a named device; simulate a FIFO worker with dynamic batching under Poisson load; report latency percentiles and sustainable throughput. End-to-end decision delay is H plus latency.
- **Output:** Score files, per-attempt outcomes, service profiles and latency tables.
- **Technical Justification:** The rigid-shift control is essential: without it, an effect could come from merely moving transactions rather than from the attack's structure. Measured service times with a validated simulator give reproducible latency estimates without long load tests.
- **Literature Support:** Blanuša et al. [15], Abolfadl [23], Heijmans et al. [53].
- **Risks & Controls:** *Risk:* the attack re-times every attempt at once, which is the most favourable case for bursts sharing background traffic. *Control:* stated explicitly; per-attempt attacks are planned. *Ethics:* synthetic data only, no real institution, and defences reported with the attack.

10.8 Step 8: Evaluation Metrics

- **Objective:** Measure detection quality, timeliness and systems behaviour.
- **Input:** Scores, labels and logs.
- **Procedure:**
  1. *Minority-class F1*, threshold chosen on validation, comparable with published work; *PR-AUC* [38] as the threshold-free metric; both reported for full test, core and tail.

2. *Stoppable value (SV).* For attempt A with first alert time $T_A = \min\{d_e : e \in A,\ s_e \geq \tau\}$,

$$\mathrm{SV}(A) = \frac{\sum_{e \in A:\ t_e > T_A} \mathrm{usd}(e)}{\sum_{e \in A} \mathrm{usd}(e)}, \tag{5}$$

and $\mathrm{SV}(A) = 0$ if no alert fires. It is reported with the transfer-count analogue, the detection rate and the median detection delay. It is an accounting measure: it assumes an alert can stop later transfers of the attempt, and it can count the same funds across hops.

3. *Alert-budget operating points:* thresholds giving B alerts/day on validation. Because alert rates shift between validation and test, results are also reported with thresholds equalised on the test window, as a sensitivity analysis that removes this bias.
4. *Attack:* detection and recall on attacked attempts; suppression relative to the matched control; attacker completion time.
5. *Systems:* service time against batch size; p50/p95/p99 latency; sustainable throughput against the benchmark's measured arrival rate.

- **Output:** Metric tables and curves generated from logs.
- **Technical Justification:** F1 credits a transfer flagged after the money is gone. Stoppable value measures what an alert can still achieve, which lets the two be compared directly.
- **Literature Support:** Saito and Rehmsmeier [38], Egressy et al. [1], Mori et al. [56].
- **Risks & Controls:** *Risk:* value share is dominated by one large transfer in an attempt. *Control:* the transfer-count analogue is always reported alongside it.

10.9 Step 9: Statistical and Comparative Analysis

- **Objective:** Quantify uncertainty and separate discovery from confirmation.
- **Input:** Per-seed scores and per-attempt outcomes.
- **Procedure:**

1. Hypotheses C1–C8 were frozen in `DECISIONS.md` before any model ran on the confirmatory datasets; HI-Small is discovery only.
2. Attempt-level metrics use a paired bootstrap over attempts (2,000 replicates), because attempts are the unit of analysis.
3. Event-level differences use a paired moving-block bootstrap over test time (1-hour blocks) [63, 64], with seeds shown individually.
4. Holm's procedure [65] is applied within each family of comparisons.
5. With n paired seeds the smallest exact two-sided Wilcoxon p-value is $2/2^n$, so we do not use it at three seeds [42].

- **Output:** Tables of differences with confidence intervals; a verdict for each hypothesis.
- **Technical Justification:** The discovery/confirmation split is the most direct guard against reporting patterns that are specific to one dataset.
- **Literature Support:** Demšar [42], Efron and Tibshirani [64], Künsch [63], Holm [65].
- **Risks & Controls:** *Risk:* few test attempts (86 on HI-Small). *Control:* rolling-origin folds and HI-Medium, which has far more attempts.

10.10 Step 10: Component and Ablation Analysis

- **Objective:** Attribute effects to specific components.
- **Input:** The modular feature set, GNN and policy.
- **Procedure:**
  1. **A1 (no forward block), A2 (no motifs), A3 (no pair block):** which evidence creates the horizon gain?
  2. **A4 (no hour of day), A5 (no payment format):** do simulator-specific attributes drive the result?
  3. **A6 (GNN, no direction separation), A7 (GNN, one hop), A8 (GNN, no cardinality term):** which parts of TD-GNN matter?
  4. **A9 (policy without re-scoring):** the policy using one score per transfer, to isolate the value of re-scoring.
  5. **A10 (wall-clock or logical-clock only):** each evidence family alone and combined, on clean and attacked data, which measures the price of timing robustness.
- **Output:** An ablation table with differences and intervals.
- **Technical Justification:** A4 and A5 are validity checks as much as ablations: an effect that vanishes without them may be a simulator artefact.
- **Literature Support:** Egressy et al. [1], Blanuša et al. [15], Yang et al. [28].
- **Risks & Controls:** *Risk:* confounding by tuning. *Control:* ablations reuse the same configuration and seeds.

10.11 Step 11: Error and Failure Analysis

- **Objective:** Understand where and why detectors fail, without feeding test information back into modelling.
- **Input:** Final predictions and per-attempt outcomes.
- **Procedure:**
  1. *Per-typology analysis:* recall, stoppable value and delay per typology against horizon.
  2. *Same-minute mechanism:* the share of illicit and benign transfers with a same-minute inflow to the sender, which explains the jump between the strict-past and same-minute views.
  3. *Non-pattern positives:* 38% of illicit HI-Small transfers belong to no listed attempt; they are analysed separately because attempt metrics cannot cover them.
  4. *Stratification:* by recipient in-degree, as in SALT-GNN [44], and by pair multiplicity.
  5. *Test-set rule:* confirmatory test sets are used once, after decisions are frozen.
- **Output:** Error-analysis tables and figures.
- **Technical Justification:** Links aggregate curves to mechanisms, which is what separates a measurement from a coincidence.
- **Literature Support:** Altman et al. [3], Losavio et al. [44].
- **Risks & Controls:** *Risk:* test-set snooping. *Control:* the discovery/confirmation split.

10.12 Step 12: Final Research Outputs

- **Objective:** Package results reproducibly.
- **Input:** Code, logs and figures.
- **Procedure:**
  1. Open-source view, feature engine, sampler, TD-GNN, policy, attack generator and queue simulator with their tests.
  2. An audit tool that reports any AMLworld-style file's day-level structure.
  3. Tables and figures generated from logs: no hand-typed numbers.
  4. The Phase 3 report and a conference paper targeting the venues in Section 15.
- **Output:** A reproducible repository and written deliverables.
- **Technical Justification:** A claim about a shared benchmark must be cheap for others to check.
- **Literature Support:** Ferretti et al. [29], Rico et al. [9], Maganti [48].
- **Risks & Controls:** *Risk:* environment drift. *Control:* pinned versions (LightGBM 4.7.0 was found to crash in this environment; 4.6.0 is pinned), seeds and dataset checksums.

11 Consolidated Methodology Summary Table

Table 8 summarises the inputs, operations and outputs of the twelve steps.

Table 8: Consolidated 12-Step Research Methodology Pipeline

Step	Stage Name	Input Data / Artifact	Core Operations / Procedures	Deliverable / Output
Step 1	Research Input	Transaction records	Directed multigraph of events; edge task; decision horizon H and view $V(e, H)$; tie semantics giving a real-time interval.	Schemas, one view definition.
Step 2	Data Collection	Public dataset releases	AMLworld HI-Small (discovery); LI-Small, HI-Medium, LI-Medium, SAML-D (confirmatory); Ethereum conditional; row counts and SHA-256 verified.	Verified raw data, data card.
Step 3	Data Preprocessing	Raw tables	Multi-GNN whole-day split reproduced; Released and Documented protocols; train-only conversion; negative sampling in training only; future-perturbation and brute-force gates.	Leakage-tested event tables.
Step 4	Feature Representation	Event tables	Past, forward, pair and motif blocks computed only from $V(e, H)$; GNN edge inputs; proposed logical-clock block.	Feature matrices per horizon.
Step 5	Proposed Methods	Views and features	Per-horizon detectors; adaptive multi-horizon alert policy with a spent budget; TD-GNN on a view-restricted sampler; timing-invariant detection; three analytical results; memory and two-stage aggregation gated.	Tested implementations.
Step 6	Baselines and Controls	Same splits and views	B0 date rule (diagnostic); B1 LightGBM; B2 official Multi-GNN; B3 TD-GNN; B4 GFP; published references; policy and attack controls.	Unified evaluation code.

Step	Stage Name	Input Data / Artifact	Core Operations / Procedures	Deliverable / Output
Step 7	Experimental Design	Detectors and protocols	7a horizon sweep; 7b benchmark audit and cadence replay; 7c adaptive alerting; 7d timing attack with matched controls, on wall-clock and logical-clock detectors; 7e service times and queue simulation.	Scores, outcomes, latency tables.
Step 8	Evaluation Metrics	Scores, labels, logs	Minority F1 and PR-AUC by region; stoppable value, transfer share, delay; budget operating points with an equal-alert sensitivity analysis; attack suppression; latency and throughput.	Generated tables and curves.
Step 9	Statistical Analysis	Per-seed and per-attempt outcomes	Frozen hypotheses C1–C8; discovery versus confirmation; paired attempt-level and block bootstraps; Holm correction.	Differences with CIs; verdicts.
Step 10	Ablation Analysis	Modular components	A1–A3 evidence blocks; A4–A5 simulator-specific attributes; A6–A8 GNN components; A9 policy without re-scoring; A10 wall-clock versus logical-clock evidence.	Ablation table.
Step 11	Error Analysis	Final predictions	Per-typology curves; same-minute mechanism; non-pattern positives; degree and multiplicity strata; test set used once.	Error-analysis report.
Step 12	Final Research Output	All artefacts	Code, audit tool and tests released; generated tables; Phase 3 report and conference paper.	Reproducible repository and reports.

12 Research Methodology Architecture Diagram

Figure 1 shows the complete research pipeline. Each box is labelled with the methodology step it implements, and its contents match Section 10. The diagram is produced from a separate TikZ source file (`methodology_architecture.tex`) and included as a vector PDF.

12.1 Architectural Narrative and Operational Band Dynamics

1. **Band A (Steps 1–2):** AMLworld (discovery and confirmatory datasets), SAML-D and, conditionally, Ethereum enter as verified event streams. The view $V(e, H)$ is defined once here.

2. **Band B (Steps 3–4):** the Multi-GNN split under the Released and Documented protocols, followed by horizon-aware features. The automated gate stops the pipeline if rewriting events outside the view changes any output.

3. **Band C (Steps 5–6):** per-horizon detectors (LightGBM and TD-GNN) feed the adaptive alert policy. The official Multi-GNN, GFP, the date rule and published values are the comparators.

4. **Band D (Step 7):** the horizon sweep, the benchmark audit with cadence replay, adaptive alerting, the timing-only attack with matched controls on wall-clock and logical-clock detectors, and service times with queue simulation, mapped to RQ1–RQ4.

5. **Band E (Steps 8–11):** metrics by test region, including stoppable value; discovery versus confirmation against frozen hypotheses; ablations that include simulator-artefact checks and the price of timing robustness; per-typology error analysis.

6. **Band F (Step 12):** code, audit tool, generated tables, the Phase 3 report and a conference paper.

13 Project Execution Plan, Feasibility, and Risk Management

13.1 Milestones

The instructor has confirmed that 15 November 2026 is a progress milestone, not the final deadline. The plan aims for credible evidence by that date, with the publication standard as the target beyond it. Table 9 lists the milestones; the detailed schedule is kept in `ROADMAP.md`.

13.2 Compute Strategy

Principle. No single machine is a dependency. Every job is a script that takes a dataset and writes a results folder, so it can run wherever capacity exists. Team laptops are for development and analysis.

Resources and allocation. Table 10 assigns each experiment to a resource. Only the official Multi-GNN strictly needs a GPU; TD-GNN benefits from one. Measured figures come from the discovery runs on an Intel i5-1235U, where a LightGBM run took about one to two minutes on HI-Small, including features. All other figures are planning estimates until measured. Kaggle's quota is commonly reported as about 30 GPU hours per week per account, which is unofficial and must be confirmed in the team's accounts. A friend's GPU is treated as extra capacity until its specification is known.

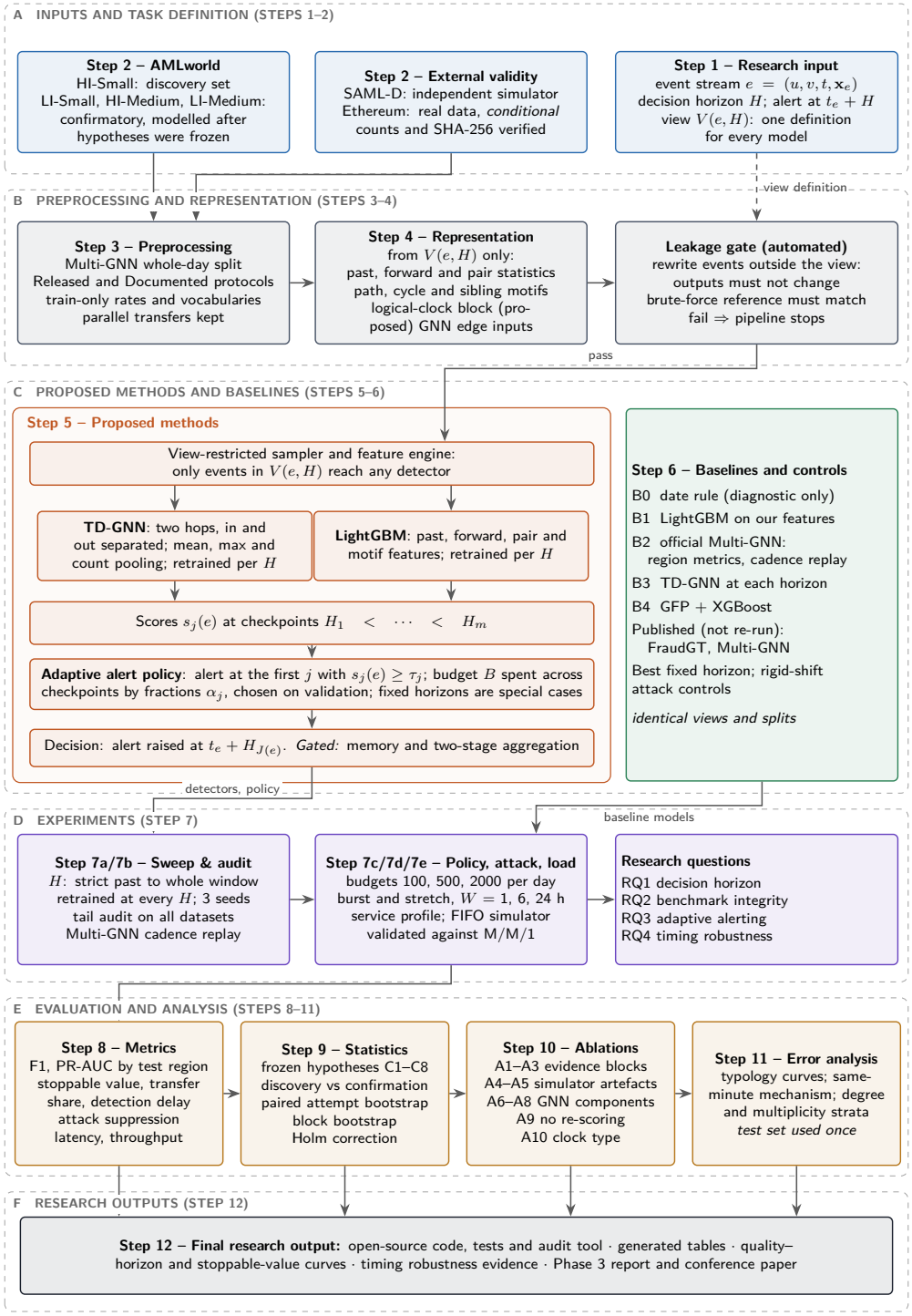


Figure 1: Research methodology architecture of the decision-horizon study. The twelve steps are organised in six bands: (A) inputs and task definition (Steps 1–2), (B) preprocessing and representation (Steps 3–4), (C) proposed methods and baselines (Steps 5–6), (D) experiments (Step 7), (E) evaluation and analysis (Steps 8–11), and (F) research outputs (Step 12). Dashed arrows denote the view definition and the frozen hypotheses, which constrain later stages without passing data.

Table 9: Milestones and Decision Gates (2026–2027). Dates after 15 November are planning targets; venue dates are in Section 15.

ID	Dates	Work	Where	Exit criterion
M0	1 Nov	HI-Small discovery: audit, main horizon sweep; hypotheses frozen	Done	Section 14
M1	2–4 Nov	Move code to a shared repository; run the leakage checks at all horizons	Kaggle CPU	All shuffled-label runs collapse to prevalence
M2	2–7 Nov	Official Multi-GNN: stock run, region metrics, cadence replay; GFP	Kaggle GPU (Multi-GNN), Kaggle CPU (GFP)	Published HI-Small F1 reproduced within its spread, or documented as not reproduced
G1	7 Nov	Does the official GNN show the tail and horizon effects?	—	Decides whether the claim covers GNNs or only our models
M3	3–10 Nov	Confirmatory sweeps; alert policy; TD-GNN at several horizons	molab or Kaggle CPU; MacBook (Medium); Kaggle or university GPU (TD-GNN)	Verdict for each frozen hypothesis
M4	5–12 Nov	Timing attack; logical-clock detector; ablations; service profile	Kaggle CPU; university GPU (latency)	Attack table with controls; price of robustness
P	**15 Nov**	**Progress milestone:** report covering all seven brief components, each marked complete or preliminary	—	Submitted
G2	late Nov	Is there a modelling gap at small horizons worth closing with memory or two-stage aggregation?	—	Build DMT-GNN only if yes
M5	Dec	Rolling-origin folds; per-attempt attacks; real Ethereum window; search repeated; maintainers contacted	All	Results robust across folds
M6	Jan–Feb 2027	Paper draft and artefact release	All	Submission if the venue's call opens

Table 10: Compute Plan by Experiment. GPU hours are estimates except where marked; the only published anchor is about 3.6 h for one Multi-GNN run on an RTX 3090 [1].

Experiment	Where	GPU hours	Notes
Official Multi-GNN: GIN, PNA, second seed	Kaggle GPU; university GPU as fallback	12–24	4–8 h per run assumed; cadence replay included; harness not yet executed
TD-GNN: 4 horizons × 3 seeds	Kaggle or university GPU; Colab overflow	12–24	Unmeasured: no full run has finished
TGN comparison, if gate G2 passes	Kaggle GPU	10–20	Gated
Service and GPU latency profile	University GPU	2–3	Needs one consistent device
LightGBM on LI-Small, attacks, ablations, leakage checks	Kaggle CPU or molab (32 GB)	0	About one to two minutes per run at HI-Small scale
LightGBM on HI-Medium, LI-Medium	MacBook (32 GB) or molab	0	About six times larger; `prepare.py` needs memory work first
GFP	Kaggle CPU (Linux)	0	Library is not supported on Windows
Total		**26–51**	**Plus 10–20 if gated work runs**

Hosted notebooks: molab. molab, the hosted notebook service of the marimo project, provides free notebooks with 4 CPUs and 32 GB of RAM and, as a public preview, an optional NVIDIA RTX Pro 6000 Blackwell GPU with 96 GB of memory; a notebook runs for at most 12 h and is shut down after more than 90 minutes of inactivity.[1] Its large memory makes it useful for the Medium datasets. We rely on it only after three acceptance tests (GPU attachment, survival of a two-hour job with the browser closed, and data-loading time). Notebooks are public by default, so no credentials may appear in notebook code.

13.3 Risk Assessment and Mitigation Strategies

Table 11 lists the main technical and scientific risks.

14 Preliminary Evidence (1 November 2026)

Every number in this section is generated from experiment logs by a script; tables and in-text values are not typed by hand. These are **discovery results on HI-Small**, except the day-level counts in Section 14.2, which cover all four datasets. They use our own detector. **The official Multi-GNN has not been run**, so statements about published models remain inferences from their code.

14.1 What Has and Has Not Been Run

Verification. All 119 automated tests pass. They include a brute-force reference for the feature engine, tests that rewrite data outside the view and require unchanged output, attack invariants, and a queue simulator check against the M/M/1 closed form. The strict-past results are identical with and without the tail days in the file, as causality requires. With shuffled training labels at strict past, core PR-AUC is 0.0012, against a prevalence of 0.0011, so the model finds nothing. *This check has not yet been completed at positive horizons, where leakage matters most; until it is, positive-horizon results carry that caveat.*

14.2 The Benchmark Tail

Table 13 and Figure 2 report the day-level audit. On every dataset, background traffic stops at the span documented by Altman et al. [3], and laundering continues. The Multi-GNN split places those days in the test set. On HI-Small, 1,108 events (59.1% illicit) hold 40.7% of the test set's illicit transfers.

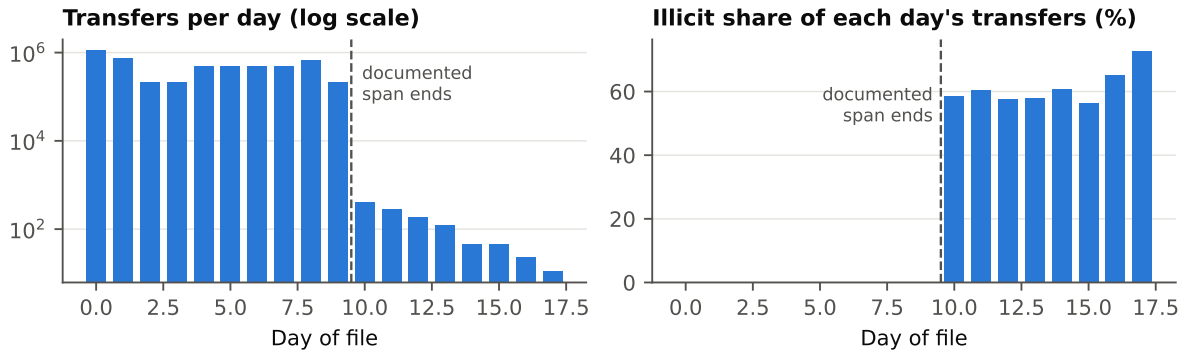


Figure 2: HI-Small by day. After the documented 10-day span, daily volume falls by about three orders of magnitude, while the illicit share rises to about 60%. Most of the test split's days lie in this region.

[1] https://docs.marimo.io/guides/molab/, accessed 25 October 2026.

Table 11: Risk Assessment and Mitigation Strategy Matrix

Identified Risk	Operational Impact	Level	Mitigation Strategy
R1: Effects are feature-model specific	The official GNN may not show the same dependence on look-ahead or tail.	High	M2 tests this directly on the official code, and gate G1 reports it either way. A narrower claim is still a valid finding.
R2: Simulator artefact	Same-minute bursts and timing may reflect AMLworld's scheduler, not laundering.	High	SAML-D as an independent simulator; hour and format ablations; real Ethereum window. No real-world effect sizes are claimed.
R3: Scooped	The audit or a timing attack appears first; Heijmans et al. (October 2026) name timestamps as future work.	Medium	Search repeated monthly and before submission; move quickly on the audit; consider a preprint.
R4: Too few attempts	86 test attempts on HI-Small give wide attempt-level intervals.	High	HI-Medium; rolling-origin folds; attempt-level bootstrap reported honestly.
R5: Logical-clock defence costs too much	Robust evidence may lose too much clean-data quality, since wall-clock gaps carry real signal.	High	This is a measured outcome (A10). A combined detector is also tested; a negative result is reported.
R6: Our GNN underperforms	TD-GNN is weak, so its horizon curve says little about GNNs.	Medium	The official Multi-GNN is the reference; TD-GNN results are reported relative to it.
R7: Metric criticism	Stoppable value assumes alerts can stop later transfers and can double count funds across hops.	Medium	Stated plainly; transfer-count analogue always reported; hop-aware variant planned.
R8: Multi-GNN audit fails to run	The harness has never been executed, and the official code may not run on current libraries.	Medium	Pinned environment on Kaggle; a time box; otherwise published values are labelled as not reproduced.
R9: Leakage bugs	Inflated horizon results.	Moderate	Future-perturbation tests, brute-force reference and mutation checks; shuffled-label runs at every horizon (M1).
R10: Compute access	Quotas or access are lower than assumed.	Moderate	Several interchangeable resources; every job is resumable by re-running one script.
R11: Responsible disclosure and ethics	Findings could be misread as an attack on a named benchmark or as evasion guidance.	Low	Synthetic data only; defences reported with the attack; dataset maintainers contacted before publication.

Table 12: Status of Experiments

Experiment	**Status**	**Note**
Tail audit, four datasets	Complete	Day-level counts only; no modelling
Horizon sweep, HI-Small, Released protocol	Complete	9 horizons, 3 seeds
Horizon sweep, HI-Small, Documented protocol	Partial	Complete to 1 h; run stopped
Shuffled-label leakage check	Partial	1 of 4 horizons
Adaptive alert policy	Partial	One seed
Feature and GNN ablations	Not run	
Timing attack	Not run	Code written and tested
TD-GNN	Not run	Code written and tested; no full run finished
Official Multi-GNN audit	Not run	Harness written but never executed
GFP; SAML-D; LI-Small and Medium models	Not run	Reserved for confirmation
Logical-clock detector	Not built	Proposed

Table 13: The post-simulation tail on AMLworld. The date rule flags every transfer in the tail. *It is a diagnostic, not a detector*: it has no learning, and it uses knowledge of the tail. Published values are from Egressy et al. [1], not re-run.

Dataset	Days in file	Documented days	Tail events	Tail illicit rate (%)	Share of test illicit (%)	Date-rule F1 (%)	XGB+GFs (publ.)	Multi-PNA+EU (publ.)
HI-Small	18	10	1,108	59.1	40.7	48.18	63.23	68.16
LI-Small	17	10	223	60.1	16.7	26.15	27.30	33.07
HI-Medium	28	16	6,987	58.5	37.1	45.37	65.70	65.67
LI-Medium	27	16	1,370	56.3	19.3	28.76	28.16	33.23

The date rule scores 48.18% F1 on HI-Small, above the 47.73% published for GIN+EU, and 28.76% on LI-Medium, above the 28.16% published for XGBoost with graph features. This does not show that those models exploit the tail. It shows that the metric's value is sensitive to a region that contains almost no negatives. How much of each published number comes from there is what the Multi-GNN audit will measure.

14.3 Quality Against Horizon

Table 14 and Figure 3 give the sweep on the discovery set, with LightGBM retrained at every horizon.

Table 14: Horizon sweep, HI-Small, Released protocol (LightGBM, mean over 3 seeds, ± s.d.). SV_B is mean stoppable value at B alerts/day with thresholds set on validation. Detection is at $B = 500$. "Core" removes the tail. Attempts: 86.

Horizon H	Test F1	Core F1	Core PR-AUC	Detected	SV_{100}	SV_{500}	SV_{2000}
strict past	54.4±0.5	37.7±0.1	0.364	93%	0.287	0.578	0.629
same minute	66.7±0.4	52.3±0.7	0.521	97%	0.319	0.617	0.634
1 min	66.6±1.2	52.5±1.6	0.526	96%	0.310	0.617	0.638
10 min	70.3±0.8	57.0±0.9	0.546	98%	0.281	0.611	0.635
1 h	73.5±0.3	62.7±0.6	0.575	95%	0.301	0.611	0.623
6 h	72.7±0.2	61.4±0.9	0.574	96%	0.276	0.570	0.589
24 h	72.9±1.1	61.5±0.5	0.572	94%	0.177	0.438	0.457
72 h	72.0±1.1	60.9±0.5	0.576	95%	0.070	0.180	0.185
whole window	71.9±0.3	61.0±0.4	0.563	94%	0.000	0.000	0.000

Four observations follow, each limited to this dataset and detector:

1. **Look-ahead has a large effect.** Test F1 rises from 54.4% (strict past) to 73.5% (1 h). On the core alone, it rises from 37.7% to 62.7%, and PR-AUC from 0.364 to 0.575.

2. **The gain plateaus near one hour.** From 1 h to the whole window, core PR-AUC changes by -0.012. The published protocol's multi-day view may add little for this detector, compared with a far shorter wait.

3. **The tail raises F1.** At the whole-window view, test F1 is 11.0 points above core F1.

4. **Real-time performance is an interval.** The strict-past and same-minute views differ in core PR-AUC by 0.157. Same-minute events have unknown order, and they matter: 39.9% of illicit core transfers have a sender receiving funds in the same minute, against 4.9% of benign ones. Funds must arrive before they are forwarded, so a real system with sub-minute ordering could see part of this. Whether this reflects laundering or the simulator's scheduler is an open question that SAML-D will help answer.

As a check on the baseline, our same-minute view (66.7% test F1) is at or above GFP's published 63.23% at batch size 128, so the causal baseline is not artificially weak. These values are not directly comparable with published GNN results: the features, thresholding and training differ.

14.4 F1 and Stoppable Value Disagree

F1 changes little from 1 h to 24 h, while stoppable value at 500 alerts/day falls from 0.611 to 0.438. The paired contrast between the best horizon of at most 1 h and 24 h is 0.179 (95% CI [0.133, 0.229]). By construction, value reaches zero at the whole window, where F1 remains

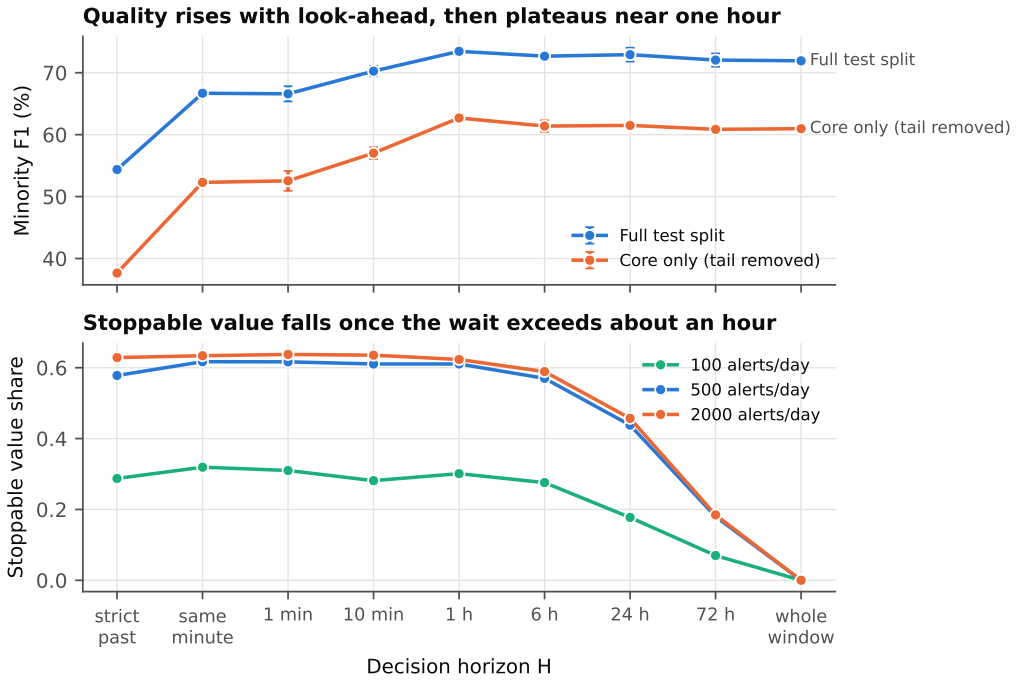


Figure 3: Quality and stoppable value against decision horizon, HI-Small, Released protocol. The two panels use separate axes because the measures differ in scale. Error bars show one s.d. over seeds. Table 14 gives the values.

high: on this measure, a whole-window evaluation describes a detector that acts only after the attempts it detects have finished.

Table 15: Stoppable value at 500 alerts/day under two threshold rules, HI-Small, Released protocol. "Val." sets thresholds on validation, as a deployment would. "Equal alerts" gives every horizon the same alert count on the test window, as a sensitivity analysis. "Transfers" is the transfer-count analogue of SV.

Horizon H	SV_{500} (val. threshold)	Transfers (val.)	SV_{500} (equal alerts)	Detected (equal alerts)
strict past	0.578	0.578	0.428	74%
same minute	0.617	0.623	0.557	89%
1 min	0.617	0.620	0.573	92%
10 min	0.611	0.617	0.576	93%
1 h	0.611	0.617	0.571	90%
6 h	0.570	0.569	0.531	90%
24 h	0.438	0.437	0.411	88%
72 h	0.180	0.175	0.152	87%
whole window	0.000	0.000	0.000	85%

A bias we corrected. Thresholds set on validation do not produce equal alert rates on test: at a nominal 500 alerts/day, the strict-past detector actually raises 944 alerts per day in the core and the 1 h detector 575. This favours strict past. With alerts equalised (Table 15), waiting is clearly better: the gain of the best short horizon (10 min) over strict past is +0.148 (95% CI [+0.095, +0.205]). **The practical reading on this dataset is to wait a little, not long.** A short wait gains evidence while most of an attempt's funds have yet to move, because attempts unfold over days. This depends on the simulator's timing: real laundering that moves faster

would shorten the useful wait, which is why the result needs independent data.

Documented protocol. Removing the tail truncates attempts at the end of the window, so stoppable value is lower throughout (Table 16), but the shape from strict past to 1 h is the same.

Table 16: Stoppable value at 500 alerts/day, HI-Small, Documented protocol. Only horizons with all three seeds complete are shown; the run was stopped before 6 h.

Horizon H	SV_{500} (val. threshold)	Transfers (val.)	SV_{500} (equal alerts)	Detected (equal alerts)
strict past	0.286	0.307	0.236	73%
same minute	0.355	0.370	0.339	90%
1 min	0.351	0.366	0.337	89%
10 min	0.354	0.369	0.347	92%
1 h	0.341	0.357	0.341	90%

By typology. Table 17 shows that the pattern holds across most typologies, though with few attempts per typology. Bipartite attempts yield the least stoppable value at every horizon shown.

Table 17: Detection and stoppable value by typology at 500 alerts/day, HI-Small. Each typology has few attempts, so differences between typologies are indicative only.

Typology	n	strict past		same minute		1 h		24 h	
		Det.	SV	Det.	SV	Det.	SV	Det.	SV
Bipartite	15	80%	0.19	87%	0.22	82%	0.21	80%	0.10
Cycle	9	100%	0.73	100%	0.77	100%	0.78	100%	0.48
Fan-In	9	93%	0.68	100%	0.70	89%	0.70	100%	0.53
Fan-Out	9	89%	0.66	89%	0.67	89%	0.67	93%	0.51
Gather-Scatter	16	98%	0.87	100%	0.85	100%	0.85	100%	0.68
Random	12	92%	0.41	100%	0.52	100%	0.52	97%	0.41
Scatter-Gather	10	100%	0.70	100%	0.73	100%	0.68	100%	0.52
Stack	6	94%	0.39	100%	0.55	100%	0.58	83%	0.24

14.5 Adaptive Alerting: Incomplete

Table 18 shows the only completed policy run, one seed of three. Adaptive allocation beats the best fixed horizon at 100 alerts/day, where the budget binds hardest; at higher budgets the interval includes zero. One seed cannot settle this, but it tempers expectations: one good fixed horizon may suffice.

Table 18: Adaptive alert policy against the best fixed horizon selected on validation, HI-Small, Released protocol. *One seed only; the run was stopped.*

Budget	Adaptive SV	Best fixed SV	Difference	95% CI
100	0.305	0.273	+0.031	[+0.008, +0.059]
500	0.623	0.615	+0.008	[-0.004, +0.025]
2000	0.640	0.636	+0.004	[-0.002, +0.014]

14.6 Hypothesis Status

Hypotheses were frozen in `DECISIONS.md` before any model ran on the confirmatory datasets. **None has yet been tested on confirmatory data.** HI-Small cannot confirm them, because it suggested them.

Table 19: Frozen Hypotheses and Current Evidence

ID	Claim	Discovery set (HI-Small)	Confirmation
C1	The tail inflates headline F1 by more than 5 points	11.0 points	Pending
C2	Look-ahead is worth more than 0.05 core PR-AUC	0.199	Pending
C3	F1 and stoppable value disagree on the best horizon	0.179, CI [0.133, 0.229]	Pending
C4	Real-time performance is an interval wider than 0.02 PR-AUC	0.157	Pending
C5	Adaptive alerting beats the best fixed horizon at $B = 500$	One seed; CI includes zero	Pending
C6	Bursts evade strict-past detectors; same-minute visibility blunts this	Not run	Pending
C7	Equal views change the GNN versus boosted-tree comparison	Not run	Pending
C8	Logical-clock evidence resists stretch attacks at acceptable clean-data cost	Not built	Pending

15 Publication Potential

15.1 Novelty and Scientific Contribution Assessment

Publication is plausible but not assured. We assess each part as a sceptical reviewer would, in descending order of current confidence:

1. **Benchmark audit (strong).** The tail is verifiable by anyone from the public files, and it affects a benchmark that recent work ranks models on. Daniluk and Dabrowski's finding on TGB [46] shows that such results are publishable. Its effect on specific published numbers still has to be shown on their code.

2. **Horizon curves and the F1–value conflict (promising).** This changes what readers should ask of a detector. Nguyen et al. [45] already show that the future helps in card fraud, so our contribution is the graph setting, the cost side of the trade-off, and equal-view comparisons. It rests so far on one dataset and one model family.

3. **Timing-only threat model (open but time-sensitive).** We found no prior timing-only attack on these models, but we have run no attack yet, and recent work names it as an extension [53].

4. **Timing-invariant detection (highest potential, least evidence).** This is the most original idea here and has real-world appeal: robustness by design, without attack examples or adversarial training. It is an untested hypothesis, and it may cost too much clean-data quality.

5. **Adaptive alerting (weak so far).** In one incomplete seed, it barely beats one good fixed horizon except at the smallest budget. If this holds, the honest and useful conclusion is that one simple short wait suffices.

6. **Modelling (deliberately de-emphasised).** DMT-GNN is incremental next to MEGA-GNN, DIAM and SALT-GNN [13, 14, 44].

Real-world impact. If confirmed, the findings would give three concrete pieces of guidance: report the decision horizon alongside any AML result; evaluate alerts by what they can still stop, not only by F1; and prefer evidence an adversary cannot change by re-timing. The first two need no new model and could be adopted immediately. All of this is on synthetic data so far, and it does not establish effect sizes in a real institution.

Minimum evidence for a credible submission. (i) The official Multi-GNN audited by region and view; (ii) horizon curves for at least two model families on at least two simulators; (iii) confirmed hypotheses on data unseen at freezing; (iv) robustness across rolling-origin folds; (v) released code and audit tool; and (vi) a repeated literature search.

15.2 Target Publication Venues

Two papers are more defensible than one overloaded paper: **Paper A** (benchmark audit, horizon framework, stoppable value) and **Paper B** (timing-only threat model and timing-invariant detection). Facts below were checked on official pages on 1 November 2026, except where marked. Rankings are not re-verified here and should be confirmed on the ICORE portal.

- **Paper A, primary: KDD 2027, Datasets and Benchmarks track** (San Jose, 1–5 August 2027). Its scope explicitly includes benchmarks, evaluation methods and auditing frameworks, with eight content pages, single-blind review and at least three reviews. Cycle 1 closed on 26 July 2026. The Research Track's page refers to a February 2027 deadline; *a second-cycle call for this track was not yet published* when checked, so this date must be confirmed.

- **Paper A, alternative: ACM ICAIF 2027.** GFP, FraudGT and LAS-GNN appeared there [15, 33, 58], so its reviewers know this benchmark. ICAIF 2026 had an August deadline, an eight-page limit including references, double-blind review and no appendices. *The 2027 call was not yet published; an August 2027 deadline is an inference.*

- **Paper B: Financial Cryptography and Data Security (FC).** FC 2027 closed on 17 September 2026 (Barbados, February 2027), with regular papers of 15 pages plus references in LNCS format, anonymised. FC 2028 is the realistic target, and only if the attack holds beyond one simulator. *Its dates are not published.*

- **Alternatives:** ECML PKDD 2027 Applied Data Science (the 2026 deadline was 12 March; 2027 dates unconfirmed); the Learning on Graphs conference (2026 deadline in August; nine-page proceedings track); the Temporal Graph Learning workshop, where Daniluk and Dabrowski appeared (2027 edition unconfirmed).

- **Not recommended now:** main research tracks of top ML venues, which expect a new method with a large effect on multiple real datasets, and networking flagships, where the systems component is too small to carry a paper on its own. Deadlines for IEEE ICC, GLOBECOM, ICBC and ACM AFT could not be confirmed from official pages and were not assessed.

Before any submission, the literature search of Section 3 is repeated, the benchmark's maintainers are contacted, and the Official Journal text of Regulation 2024/886 is checked directly.

16 Conclusion

This document extends the Phase 1 review with nine verified 2025–2026 studies from Q1/Q2 journals and CORE A* conferences, positions the project against the closest earlier work, and turns the resulting gaps into a twelve-step methodology.

Revision 3 changes the direction of the project on the basis of direct evidence. Reading the released code shows that the most-cited AML results use an unstated, non-uniform view of future transactions. Reading the raw data shows that the standard test split extends past the end of background traffic on all four datasets we checked. Preliminary measurements on the discovery set suggest that strict causality costs a large share of detection quality, that the gain from looking ahead plateaus near an hour for our feature-based detector, and that F1 and stoppable value disagree about how long to wait: a short wait pays, a long one does not. These are discovery results on one dataset and one family of models. The official GNN has not yet been run, and the hypotheses that would make these results general are frozen and untested.

The Computer Networks contribution lies in treating detection as a delay–information trade-off in a stream-processing service, and in proposing logical rather than physical time as the basis for evidence an adversary cannot move. The project's aim remains a defensible conference paper; 15 November 2026 is the first evidence-of-progress milestone on the way.

A Requirement Traceability Matrix

Table 20 maps each Phase 2 requirement to where it is addressed.

Table 20: Phase 2 Requirement Traceability Matrix

Requirement	How it is addressed	Location
8–10 additional papers, 2025–2026	Nine papers; Phase 1 papers excluded; years verified on Crossref	Section 3, Table 2
Genuine research articles in Q1/Q2 or ranked venues	Seven journal articles (Q1/Q2) and two CORE A* conference papers; sources in Appendix B	Table 2
Relevance, methodology, data and evaluation	Fifteen headings (A–O) per paper	Section 4
Screening transparency	Excluded candidates with reasons	Table 3
Cross-paper comparison table (title, year, dataset, preprocessing, models, metrics, results, limitations)	Landscape table with split protocol stated; capability matrix	Tables 5, 6
At least three research gaps with evidence from three papers	Six gaps (G4 deferred; G5 and G6 added in Revision 3), evidence from KAR, LMAE4Eth, TAML and the code audits	Table 7, Section 8.1
Refined gap answering six questions	Six questions; gap statement, problem, aim, objectives, RQ1–RQ4	Section 9
Twelve-step methodology	Steps 1–12 with objective, input, procedure, output, justification, literature, risks	Section 10, Table 8
Methodology architecture diagram	Vector TikZ figure consistent with Steps 1–12	Figure 1
Feasibility within the project timeline	Revision 3: milestones and gates to the 15 November progress date and beyond; CPU-first compute strategy; risks	Section 13, Tables 9, 11
Publication potential	Contribution assessment and venue fit	Section 15
Closest prior work	Multi-GNN, DIAM, MEGA-GNN, GFP, FraudGT (with reported results), SALT-GNN, PRES, BADGNN, AnchorRM, StreamTGN, TeMP-TraG, Amatriciana; added in Revision 3: KONTOGRAPH, Maganti, MemFreezing, HIA, TDAP, Heijmans et al., LAS-GNN	Section 5, Table 4

B Literature Verification Log

Bibliographic metadata were checked against Crossref (`api.crossref.org`) and the publisher landing pages in October 2026. Conference ranks were checked on the ICORE portal. The SCImago portal blocked automated access, so journal quartiles come from publisher bibliometric pages or secondary SJR/JCR listings and should be re-confirmed on https://www.scimagojr.com before final submission. The entries added in Revision 2 (below the FraudGT row) come from arXiv abstract pages and, for StreamTGN, search-result excerpts; at that time their full texts were still to be read, and none of them is counted towards the nine-paper quota. On 1 November 2026, the full texts of AnchorRM, StreamTGN, MemFreezing, HIA, KONTOGRAPH, Maganti and Heijmans et al. and the relevant sections of Multi-GNN, MEGA-GNN, GFP and SALT-GNN were read; the Multi-GNN repository (commit `252b025`) and FraudGT's dataset class were inspected directly. LAS-GNN and TDAP were checked against Crossref metadata only; LAS-GNN's full text was not accessible. Regulation 2024/886 was cited from secondary sources because EUR-Lex could not be read automatically. Searches that found no prior work on the AMLworld tail, on horizon curves or on timing-only attacks do not prove none exists.

References

[1] B. Egressy, L. von Niederhäusern, J. Blanuša, E. Altman, R. Wattenhofer, and K. Atasu, "Provably powerful graph neural networks for directed multigraphs," in *Proc. AAAI Conference on Artificial Intelligence*, vol. 38, no. 10, 2024, pp. 11 838–11 846.

[2] Chainalysis, "Crypto crime reaches record high in 2025 as nation-state sanctions evasion moves on-chain at scale (The 2026 crypto crime report: Introduction)," Chainalysis Inc., Jan. 2026, published 8 January 2026; lower-bound estimate. [Online]. Available: https://www.chainalysis.com/blog/2026-crypto-crime-report-introduction/.

[3] E. Altman, J. Blanuša, L. von Niederhäusern, B. Egressy, A. Anghel, and K. Atasu, "Realistic synthetic financial transactions for anti-money laundering models," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, vol. 36, 2023, pp. 29 851–29 874.

[4] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. 5th International Conference on Learning Representations (ICLR)*, 2017.

[5] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[6] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. 6th International Conference on Learning Representations (ICLR)*, 2018.

[7] Z. Tian and D. Zhang, "Continual graph learning with knowledge-augmented replay: A case for Ethereum phishing detection," *Electronics*, vol. 14, no. 17, p. Art. no. 3345, 2025.

[8] S. Al-E'mari, Y. Sanjalawe, S. Fraihat, and A. Aljarf, "Explainable graph neural networks for multi-class fraudulent address classification in Ethereum," *Future Internet*, vol. 18, no. 9, p. Art. no. 461, 2026.

[9] R. Rico, I. Hulpus, S. Leisink, B. Zhao, and Y. Velegrakis, "Translating latent representations for money laundering detection," in *Proc. 35th International Joint Conference on Artificial Intelligence (IJCAI)*, 2026, pp. 6500–6508.

Table 21: Literature Verification Log

Key	First author, year, venue	DOI / identifier	Ranking (source)	Full text examined; notes
yang2025	X. Yang, 2025, AAAI-25	10.1609/aaai.v39i20.35508	CORE2023 A* (ICORE)	arXiv:2412.16447; hyperparameters not reported, so omitted.
tian2025	Z. Tian, 2025, Electronics	10.3390/electronics14173345	SJR Q2 (secondary)	MDPI HTML; result-table values not extracted, so only text-stated results are quoted.
ferretti2025	S. Ferretti, 2025, IEEE Access	10.1109/ACCESS.2025.3552240	SJR Q1 (secondary); SJR 2024 0.849 (IEEE)	IEEE Xplore HTML; split protocol not described.
li2025	M. Li, 2025, Sci. Rep.	10.1038/s41598-025-08365-9	JCR Q1 (secondary)	Nature HTML and tables; split not described; baselines borrowed.
kasula2025	V. K. Kasula, 2025, Discover AI	10.1007/s44163-025-00508-1	SJR Q1, 0.876 (secondary)	Springer HTML and tables.
jia2025	Y. Jia, 2025, IEEE TIFS	10.1109/TIFS.2025.3612149	SJR Q1 (secondary)	Authors' version arXiv:2509.03939v1; may differ slightly from the version of record.
lin2026	Z. Lin, 2026, Sci. Rep.	10.1038/s41598-025-23901-3	JCR Q1 (secondary)	Nature HTML; dataset statistics differ from canonical Elliptic.
alemari2026	S. Al-E'mari, 2026, Future Internet	10.3390/fi18090461	SJR Q2 (secondary)	MDPI HTML and tables; adjusted $p = 0.074$ (Table 12 of the paper).
rico2026	R. Rico, 2026, IJCAI-26	10.24963/ijcai.2026/723	CORE2023 A* (ICORE)	Proceedings PDF.
egressy2024	B. Egressy, 2024, AAAI-24	10.1609/aaai.v38i10.29069	CORE2023 A* (ICORE)	arXiv v3 tables and appendix (F1 values, training times, throughput).
ding2024	Z. Ding, 2024, CIKM-24	10.1145/3627673.3679707	CORE2023 A (ICORE)	Abstract and arXiv record only.
bilgi2026	H. Ç. Bilgi, 2026, TMLR	arXiv:2412.00241v3	Not verified	Acceptance as stated on arXiv; abstract only.
blanusa2024	J. Blanuša, 2024, ICAIF-24	10.1145/3677052.3698674	Not checked	arXiv:2402.08593v2 read 1 Nov 2026; Table 5 inspected as a rendered page.
lin2024	J. Lin, 2024, ICAIF-24	10.1145/3677052.3698648	Not checked	Revision 2: author PDF, Tables 1–2 (dataset statistics, split, minority F1).
losavio2026	L. Losavio, 2026, arXiv	arXiv:2607.10131v1	Preprint	Abstract page only (July 2026); code not yet released.
su2024	J. Su, 2024, ICLR-24	arXiv:2402.04284	Not checked	Abstract only.
zhou2025b	Y. Zhou, 2025, arXiv	arXiv:2506.19282v1	Preprint	Abstract only; training-focused.
anchorrm	H. Gharwi, 2026, arXiv	arXiv:2610.27558v1	Preprint	Full text read 1 Nov 2026.
streamtgn	L. Zhang, 2026, arXiv	arXiv:2603.21090v1	Preprint	Full text read 1 Nov 2026.
dai2025	Y. Dai, 2025, ICML	PMLR 267:12121–12160	Not checked	Full text read 1 Nov 2026; threat model verified.
heijmans2026	D. Heijmans, 2026, arXiv	arXiv:2610.31718v1	Preprint	HTML read 1 Nov 2026, except the final part of Appendix G.
abolfadl2026	A. Abolfadl, 2026, arXiv	arXiv:2608.22389v1	Preprint	Full text read 1 Nov 2026; not peer reviewed.
maganti2026	S. Maganti, 2026, arXiv	arXiv:2604.19514v1	Preprint	Read 1 Nov 2026; not peer reviewed.
jeon2025	D. H. Jeon, 2025, arXiv	arXiv:2509.25418v1	Preprint	Threat model read 1 Nov 2026.
sharma2023	K. Sharma, 2023, KDD	10.1145/3580305.3599517	Not checked	Crossref metadata and search excerpt only.
verlaan2025	S. Verlaan, 2025, ICAIF	10.1145/3768292.3770410	Not checked	Crossref metadata only; full text inaccessible (HTTP 403).
oztas2023	B. Oztas, 2023, ICEBE	Crossref title match	Not checked	Metadata only; dataset not yet obtained.
chen2025	S. Chen, 2025, Adv. Intell. Syst.	10.1002/aisy.202400898	Q1 (secondary)	Excluded: full text behind a bot-verification page; abstract only.
zhou2025	W.-G. Zhou, 2025, IEEE Access	10.1109/ACCESS.2025.3553377	Q1 (secondary)	Excluded: non-financial datasets.
asiri2025	A. Asiri, 2025, Sci. Rep.	10.1038/s41598-025-95672-w	JCR Q1 (secondary)	Excluded: anomalies confirmed in the PubMed Central full text.

[10] Y. Jia, Y. Wang, J. Sun, Y. Tian, and P. Qian, "LMAE4Eth: Generalizable and robust Ethereum fraud detection by exploring transaction semantics and masked graph embedding," *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 10 260–10 274, 2025.

[11] V. K. Kasula, A. R. Yadulla, M. Yenugula, B. Konda, and S. Ayyamgari, "Blockchain address classification using graph neural networks for enhanced security and analytics," *Discover Artificial Intelligence*, vol. 5, p. Art. no. 285, 2025.

[12] M. Li, L. Jia, and X. Su, "Global-local graph attention with cyclic pseudo-labels for Bitcoin anti-money laundering detection," *Scientific Reports*, vol. 15, p. Art. no. 22668, 2025.

[13] Z. Ding, J. Shi, Q. Li, and J. Cao, "Effective illicit account detection on large cryptocurrency MultiGraphs," in *Proc. 33rd ACM International Conference on Information and Knowledge Management (CIKM)*, 2024, pp. 457–466.

[14] H. Ç. Bilgi and K. Atasu, "MEGA: Message passing neural networks for multigraphs with edge attributes," arXiv preprint arXiv:2412.00241v3; listed by the authors as accepted for publication in Transactions on Machine Learning Research (TMLR), 2026, 2026.

[15] J. Blanuša, M. C. Baraja, A. Anghel, L. von Niederhäusern, E. Altman, H. Pozidis, and K. Atasu, "Graph feature preprocessor: Real-time subgraph-based feature extraction for financial crime detection," in *Proc. 5th ACM International Conference on AI in Finance (ICAIF)*, 2024, pp. 222–230.

[16] J. Su, D. Zou, and C. Wu, "PRES: Toward scalable memory-based dynamic graph neural networks," in *Proc. International Conference on Learning Representations (ICLR)*, 2024, arXiv:2402.04284.

[17] Y. Zhou and X. Ren, "A batch-insensitive dynamic GNN approach to address temporal discontinuity in graph streams," arXiv preprint arXiv:2506.19282, 2025.

[18] H. Gharwi, K. Shu, and Y. Dai, "AnchorRM: Parallel intra-batch state reconstruction with anchor-based residual memory for temporal graph neural networks," arXiv preprint arXiv:2610.27558, 2026.

[19] L. Zhang, P. Qiao, Z. Zhang, Y. Yuan, and G. Wang, "StreamTGN: A GPU-efficient serving system for streaming temporal graph neural networks," arXiv preprint arXiv:2603.21090, 2026.

[20] S. Kaul, R. Yates, and M. Gruteser, "Real-time status: How often should one update?" in *2012 Proceedings IEEE INFOCOM*, 2012, pp. 2731–2735.

[21] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.

[22] European Parliament and Council of the European Union, "Regulation (EU) 2024/886 on instant credit transfers in euro," Official Journal of the European Union, 2024, requirements cited via secondary sources; the Official Journal text could not be retrieved automatically and should be checked before submission.

[23] A. Abolfadl, "KONTOGRAPH: Verified point-in-time feature consistency and amortised explanation for real-time anti-money laundering under a 200 ms decision budget," arXiv preprint arXiv:2608.22389, Aug. 2026.

[24] C. Li, Y. Li, Q. Song, J. Gao, H. Duan, and Q. Zeng, "TcfDyG: Cryptocurrency network anomaly detection based on time-aware channel fusion dynamic graph neural network," *Blockchain: Research and Applications*, p. Art. no. 100553, 2026.

[25] J. Lu, B. Liao, Y. Liu, and L. Zhong, "ETX2Vec: A fraud detection algorithm for Ethereum based on temporal biased random walk strategy," *Scientific Reports*, vol. 16, p. Art. no. 16743, 2026.

[26] N. Han, R. Zhang, X. Liu, and H. Zhang, "Illicit Bitcoin transaction detection via feature-gated temporal graph learning," *Scientific Reports*, vol. 16, p. Art. no. 22352, 2026.

[27] Z. Lin, Q. Luo, D. Wu, J. Shen, L. Li, X. Nong, and Z. Qin, "Detecting illicit transactions in Bitcoin: A wavelet-temporal graph transformer approach for anti-money laundering," *Scientific Reports*, vol. 16, p. Art. no. 1548, 2026.

[28] X. Yang, X. Zhao, and Z. Shen, "A generalizable anomaly detection method in dynamic graphs," in *Proc. AAAI Conference on Artificial Intelligence*, vol. 39, no. 20, 2025, pp. 22 001–22 009.

[29] S. Ferretti, G. D'Angelo, and V. Ghini, "Enhancing anti-money laundering frameworks: An application of graph neural networks in cryptocurrency transaction classification," *IEEE Access*, vol. 13, pp. 50 201–50 215, 2025.

[30] S. Chen, Y. Liu, Q. Zhang, Z. Shao, and Z. Wang, "Multi-distance spatial-temporal graph neural network for anomaly detection in blockchain transactions," *Advanced Intelligent Systems*, vol. 7, no. 8, p. Art. no. 2400898, 2025.

[31] W.-G. Zhou and K. Abbas, "Unsupervised inductive node representation learning for dynamic graphs," *IEEE Access*, vol. 13, pp. 55 034–55 048, 2025.

[32] A. Asiri and K. Somasundaram, "Graph convolution network for fraud detection in Bitcoin transactions," *Scientific Reports*, vol. 15, p. Art. no. 11076, 2025.

[33] J. Lin, X. Guo, Y. Zhu, S. Mitchell, E. Altman, and J. Shun, "FraudGT: A simple, effective, and efficient graph transformer for financial fraud detection," in *Proc. 5th ACM International Conference on AI in Finance (ICAIF)*, 2024, pp. 292–300.

[34] S. Gounoue, A. Sao, and S. Gottschalk, "TeMP-TraG: Edge-based temporal message passing in transaction graphs," arXiv preprint arXiv:2503.16901, 2025.

[35] M. D. Gennaro, F. Panebianco, M. Pianta, S. Zanero, and M. Carminati, "Amatriciana: Exploiting temporal GNNs for robust and efficient money laundering detection," arXiv preprint arXiv:2506.00654 (journal reference listed on arXiv: IEEE ICDM Workshops 2024), 2025.

[36] D. Xu, C. Ruan, E. Korpeoglu, S. Kumar, and K. Achan, "Inductive representation learning on temporal graphs," in *Proc. 8th International Conference on Learning Representations (ICLR)*, 2020.

[37] E. Rossi, B. Chamberlain, F. Frasca, D. Eynard, F. Monti, and M. Bronstein, "Temporal graph networks for deep learning on dynamic graphs," arXiv preprint arXiv:2006.10637, 2020.

[38] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLOS ONE*, vol. 10, no. 3, p. e0118432, 2015.

[39] M. Weber, G. Domeniconi, J. Chen, D. K. I. Weidele, C. Bellei, T. Robinson, and C. E. Leiserson, "Anti-money laundering in Bitcoin: Experimenting with graph convolutional networks for financial forensics," arXiv preprint arXiv:1908.02591 (KDD 2019 Workshop on Anomaly Detection in Finance), 2019.

[40] K. Xu, C. Li, Y. Tian, T. Sonobe, K. ichi Kawarabayashi, and S. Jegelka, "Representation learning on graphs with jumping knowledge networks," in *Proc. 35th International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 80, 2018, pp. 5453–5462.

[41] R. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec, "GNNExplainer: Generating explanations for graph neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[42] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.

[43] Y. Elmougy and L. Liu, "Demystifying fraudulent transactions and illicit nodes in the Bitcoin network for financial forensics," in *Proc. 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2023, pp. 3979–3990.

[44] L. Losavio, F. Sovrano, D. Fenoglio, M. Gjoreski, and M. Langheinrich, "SALT-GNN: Handling dense neighborhoods in anti-money laundering graphs via statistics-aware attention," arXiv preprint arXiv:2607.10131, 2026.

[45] V. B. Nguyen, K. G. Dastidar, M. Granitzer, and W. Siblini, "The importance of future information in credit card fraud detection," in *Proceedings of the 25th International Conference on Artificial Intelligence and Statistics (AISTATS)*, ser. PMLR, vol. 151, 2022, arXiv:2204.05265.

[46] M. Daniluk and J. Dabrowski, "Temporal graph models fail to capture global temporal dynamics," in *Temporal Graph Learning Workshop at NeurIPS*, 2023, arXiv:2309.15730.

[47] J. Su and S. Wu, "Temporal-aware evaluation and learning for temporal graph neural networks," arXiv preprint arXiv:2412.07273, 2024.

[48] S. Maganti, "When graph structure becomes a liability: A critical re-evaluation of graph neural networks for Bitcoin fraud detection under temporal distribution shift," arXiv preprint arXiv:2604.19514, Apr. 2026.

[49] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, pp. 1–21, 2012.

[50] Y. Dai, L. Liu, X. Tang, Y. Zhang, and J. Yang, "MemFreezing: A novel adversarial attack on temporal graph neural networks under limited future knowledge," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. PMLR, vol. 267, 2025, pp. 12 121–12 160.

[51] D. H. Jeon, L. Zhu, H. Li, P. Li, J. Feng, T. Duan *et al.*, "Leveraging vulnerabilities in temporal graph neural networks via strategic high-impact assaults," arXiv preprint arXiv:2509.25418, Sep. 2025.

[52] K. Sharma, R. Trivedi, R. Sridhar, and S. Kumar, "Temporal dynamics-aware adversarial attacks on discrete-time dynamic graph models," in *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2023, pp. 2023–2035.

[53] D. Heijmans, H. Ç. Bilgi, R. Wang, and K. Atasu, "Towards adversarially robust multigraph neural networks for financial crime analysis," arXiv preprint arXiv:2610.31718, Oct. 2026.

[54] C. Doytshman, S. Momiyama, I. Singh, Y. Elovici, and A. Shabtai, "FRAUDability: Estimating users' susceptibility to financial fraud using adversarial machine learning," arXiv preprint arXiv:2312.01200, 2023.

[55] H. Shen and E. Kurshan, "Deep Q-network-based adaptive alert threshold selection policy for payment fraud systems in retail banking," in *ACM International Conference on AI in Finance (ICAIF)*, 2020.

[56] U. Mori, A. Mendiburu, S. Dasgupta, and J. A. Lozano, "Early classification of time series by simultaneously optimizing the accuracy and earliness," *IEEE Transactions on Neural Networks and Learning Systems*, 2018.

[57] K. K. G. Lan and D. L. DeMets, "Discrete sequential boundaries for clinical trials," *Biometrika*, vol. 70, no. 3, pp. 659–663, 1983.

[58] S. Verlaan, I. Hulpuş, and E. J. van Leeuwen, "LAS-GNN: A graph neural network for temporal money laundering motif detection," in *Proceedings of the 6th ACM International Conference on AI in Finance*, 2025, pp. 256–264.

[59] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2980–2988.

[60] B. Oztas, D. Cetinkaya, F. Adedoyin, M. Budka, H. Dogan, and G. Aksu, "Enhancing anti-money laundering: Development of a synthetic transaction monitoring dataset," in *2023 IEEE International Conference on e-Business Engineering (ICEBE)*, 2023.

[61] J. Wu, Q. Yuan, D. Lin, W. You, W. Chen, C. Chen, and Z. Zheng, "Who are the phishers? Phishing scam detection on Ethereum via network embedding," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 52, no. 2, pp. 1156–1166, 2022.

[62] M. van den Beukel, J. M. Rozanec, and A.-L. Varbanescu, "Tide: A customisable dataset generator for anti-money laundering research," arXiv preprint arXiv:2603.01863, Mar. 2026.

[63] H. R. Künsch, "The jackknife and the bootstrap for general stationary observations," *The Annals of Statistics*, vol. 17, no. 3, pp. 1217–1241, 1989.

[64] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall/CRC, 1993.

[65] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.